

1. 软件工程发展

- 软件开发的四大本质难题：不可见性、复杂性、一致性、可变性。
 - 除了不可见性以外，其他三个本质难题因项目而异
 - 四大本质难题互动促进，可以缓解，但是不能彻底解决
- 软件危机：落后的软件生产方式无法满足迅速增长的计算机软件需求，从而导致软件开发与维护过程中出现一系列严重问题的现象。
- 软件发展的两个趋势：软件项目规模日益扩大、软件在整个系统的比重日益增加。
- 软件发展的三大阶段：
 1. 软硬件一体化 50年代 到 70年代
 1. 特点：软件支持硬件完成计算任务、功能单一、复杂度有限、几乎没有需求变更
 2. 开发方法：线性顺序过程、三思而后行(Measure twice, and cut once)、编码和改错(Code and Fix)
 2. 软件成为独立的产品 70年代 到 90年代
 1. 特点：软件摆脱硬件的束缚(操作系统)、功能强大(个人电脑出现)、需求多变、兼容性要求、来自市场的压力
 2. 开发方法：形式化方法、结构化程序设计和瀑布模型
 3. 网络化和服务化：90年代中期 至今
 1. 特点：功能更复杂、规模更大、用户数量急剧增加、快速演化和需求不确定、分发方式的变化、进一步的服务化和网络化、盛行开源和共享精神。
 2. 开发方法：迭代式开发、敏捷开发(XP、Scrum、KanBan)、开源软件开发方式、DevOps

2. 软件工程与管理

- 软件工程是一门研究用工程化的方法构建和维护有效的、实用的和高质量的软件的学科。
 - 软件工程的管理视角：能否复制成功？
 - 软件工程的技术视角：能够将问题解决得更好？

2.1. 管理

- 管理的三大关键要素：目标、状态、纠偏
 - 软件项目管理是为了较少各种无谓的损耗来实现本该有的性能。
 - 软件过程改进是为了达到更好的效能，其中质量(缺陷)是首要目标或限制。
- 软件项目管理是应用方法、工具、技术以及人员能力来完成软件项目，实现项目目标的过程。
 - 软件项目管理的三大目标：成本、质量、工期。
 - 软件项目管理的对象是各项软件项目。
 - 软件项目管理的管理视角可以细分为软件过程和生命周期模型。
- 软件过程管理
 - 软件过程管理的目的是为了让软件过程在开发效率、质量等方面有着更好性能绩效
 - 软件过程管理的对象是软件过程。

- 软件过程管理与软件过程改进
 - 两者含义接近
 - 软件过程管理参考模型 CMM/CMMI、SPICE等
 - 软件过程改进参考元模型 PDCA、IDEAL等

2.2. 软件过程

- 软件过程：软件过程是为了实现一个或者多个事先定义的目标而建立起来的一组实践(具有一定的先后顺序)的集合。

2.2.1. 广义软件过程

- 广义软件过程(同义词：软件开发方法、软件开发过程)
 - 包括技术、人员及狭义过程。
 - 例子：净室Clean Room方法、极限编程方法、Scrum方法、Gate方法、敏捷软件过程/方法、轻量型过程/方法和重量型软件过程/方法。

2.2.2. 生命周期模型

- 生命周期模型：瀑布模型、迭代式模型、增量模型、螺旋模型、原型法。
- 生命周期模型与软件过程的区别和联系
 - 生命周期模型是对软件过程的一种人为划分。
 - 生命周期模型是软件开发过程的框架，是对软件开发过程的一种粗粒度划分。
 - 生命周期模型往往不包括技术实践。
- 如何理解瀑布模型
 - 瀑布模型不是单一模型，是一系列模型，覆盖最简单场景(过程元素少)到最复杂的场景(过程元素多)
 - 软件项目应该结合实际情况选择合适过程元素的瀑布模型。基本原则是，项目面临困难和挑战越多，选择的模型应该越复杂。
 - 软件项目团队往往低估项目的挑战，选择了过于简单的不适用的瀑布模型。

2.2.3. 敏捷软件开发

- 目的：使软件开发团队具有高效工作和快速响应变化的能力
- 特征：小周期迭代、快速响应变更、价值交付、自动化
- 雪鸟会议与敏捷宣言
 - 敏捷软件开发宣言的4个简单价值观
 - 个体和交互 胜过 过程和工具
 - 可以工作的软件 胜过 面面俱到的文档
 - 客户合作 胜过 合同谈判
 - 响应变化 胜过 遵循计划
 - 也就是说，尽管右项有其价值，我们更重视左项的价值
- 具有误导性的描述
 - 轻量级方法：对XP为代表的的一类方法的误导，轻量级和重量级没有具体的区分方法。
 - 拥抱变更、变更驱动：对待变更，所有的软件工程方法都是限制和管理的态度。所有正式的项目都是计划驱动的，否则计划的作用无法体现。
 - 测试驱动开发可以提供更高的开发质量：没有足够的证据支持

2.2.4. 敏捷软件开发方法

- 极限编程方法 XP
 - 将好的开发实践运用到极致，规定开发人员每周工作时间不超过40小时，连续加班不可以超过2周。
 - 比如客户作为开发团队的成员、短交付周期、结对编程、测试驱动开发(TDD)、持续集成、重构
- Scrum：
 - 3个角色：产品负责人、Scrum Master、开发团队
 - 3个工件：产品订单(优先级、动态)、冲刺订单(细化、冻结)、燃尽图
 - 5个活动：Sprint、Sprint Planning、Sprint Daily Standup、Spring Review、Sprint Retrospective
 - 5个价值观：Coverage(勇气)、Openness(开放)、Focus(专注)、Commitment(承诺)、Respect(尊重)
- KanBan方法
 - 精益生产(丰田制造法)的具体实现
 - 活动：可视化 workflow、限制WIP、管理周期时间
- Scrum vs. XP
 - 迭代周期不同：Scrum的迭代周期为2-4周，XP的迭代周期为1-2周
 - 迭代中是否允许需求变更：Scrum迭代中需求冻结，XP只要时间资源相同即可变更。
 - 迭代中是否严格遵守需求优先级：Scrum比较灵活(可能有需求依赖)，XP严格遵守。
 - 过程工程化：Scrum开发过程并未工程化，XP对开发流程严格定义。
- Linus 定律：如果有足够多的beta测试者和合作开发者，几乎所有问题都会很快显现，然后自然有人会把它解决。

2.3. 迭代式开发方法

1. 定义：大型软件系统的开发过程也是一个逐步学习和交流的过程，软件系统的交付不是一次完成，而是通过多个迭代周期，逐步来交付。
2. 特点和优势：

2.4. 题目

- 软件过程管理是软件项目管理应该要实现的目标：错误
- 在公司导入敏捷过程是我们今年过程改进的目标：正确
- XP与CMM/CMMI是对立的两种软件开发方法：正确
- CMM/CMMI不适合当今互联网环境的项目管理需求：正确
- PDCA和IDEAL不适合在敏捷环境中使用：错误
- 不同的软件开发案过程应该使用不同的生命周期模型，反之亦然：错误
- 【2016】Devops三个步骤
- 【2018】Devops的特点，为什么流行？
- 【2018】什么是云原生？相关的重要的概念
- 【2018】精益屋的两大支柱？
- 【2018】JIT及时生产，价值流和价值拉动的关系

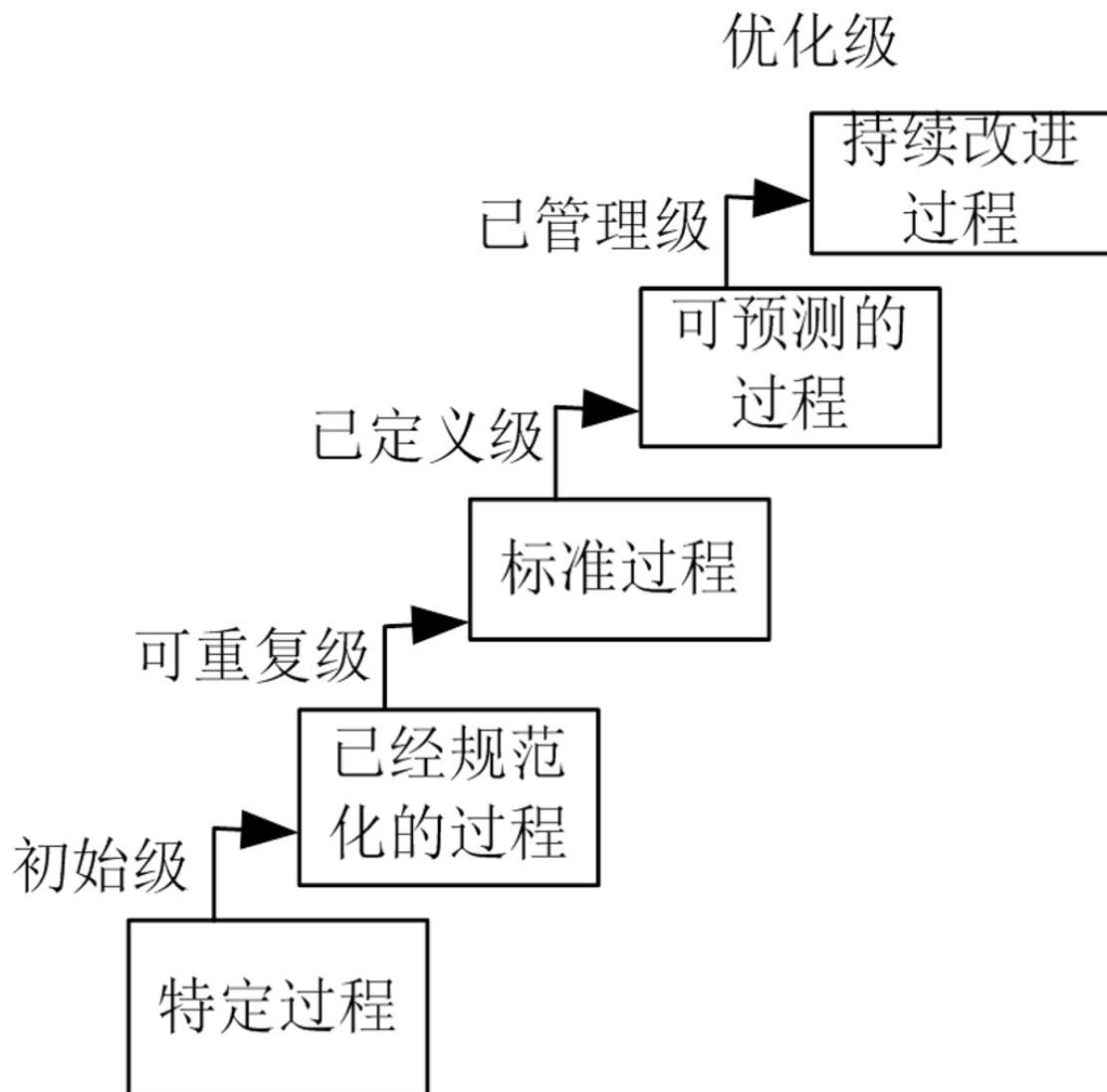
3. 各类模型

3.1. CMM：软件过程管理/改进模型，Humphrey

1. CMM是一种用于评价软件承包能力并帮助其改善软件质量的方法，侧重于软件开发过程的管理及工程能力的提高与评估。

2. 五个等级

1. 初始级 Initial：几乎没有健全的软件工程管理制度
2. 可重复级 Repeatable：有基本的软件项目的管理行为，设计和管理技术是基于相似产品中的经验，复用以前的成功。
3. 已定义级 Defined：为软件生产的过程编制了完整的文档。
4. 已管理级 Managed：公司为每个项目都设定质量和生产目标。
5. 优化级 Optimizing：连续地改进软件过程。



	等级	特征	主要需解决的问题
五	优化级	经反馈得以改进的过程	保持优化的机构，但仍为人员密集的过程
四	已管理级	(量化的)已度量的过程	技术变更、问题分析、问题预防
三	已定义级	(量化的)已定义且制度化的过程	过程度量、过程分析、量化质量计划
二	可重复级	(直觉的)过程依赖于个人	培训、测试、技术常规和评审、过程关注、标准和过程
一	初始级	个别的、混乱的过程	项目管理、项目策划、配置管理、软件质量保证

3.2. CMMI：软件过程管理/改进模型

1. CMMI刻画软件团队/组织从不成熟到成熟的每个阶段的特征

2. 五个等级

1. 初始级 Initial

1. 过程不可预测、项目管理很少、开发相对混乱
2. 个人英雄主义、救火文化

2. 已管理级 Managed

1. 以项目为单位进行管理，相对被动的管理
2. 有项目计划和跟踪、需求管理、配置管理等

3. 已定义级 Defined

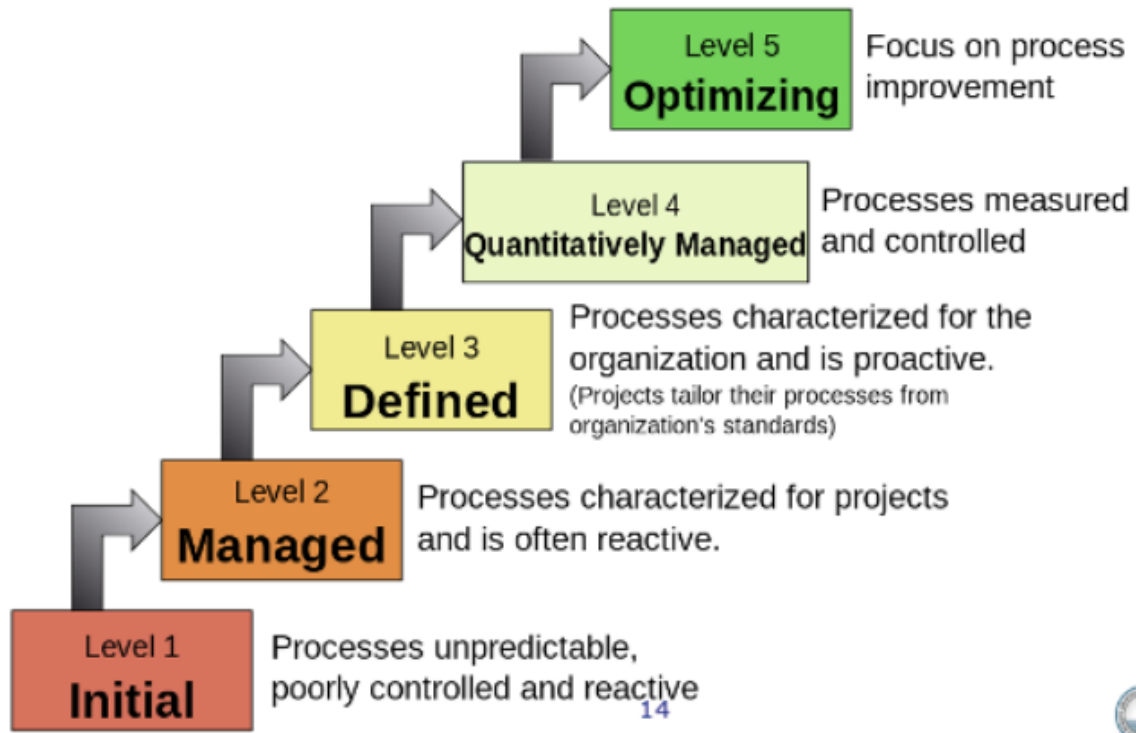
1. 以公司为单位进行管理，相对主动的管理
2. 公司层面有标准流程和相应规范，每个项目小组可以基于此定义自己的流程

4. 定量管理级 Quantitatively

1. 过程被度量和管理的
2. 构建预测模型，用统计过程控制的手段来管理过程

5. 优化级 Optimizing

1. 关注与过程改进
2. 继续应用统计方法识别过程偏差，找到问题根源并消除，避免未来继续发生类似问题



3.3. PDCA：软件过程改进模型

1. PDCA：Plan、Do、Check、Action

2. PDCA模型的步骤：

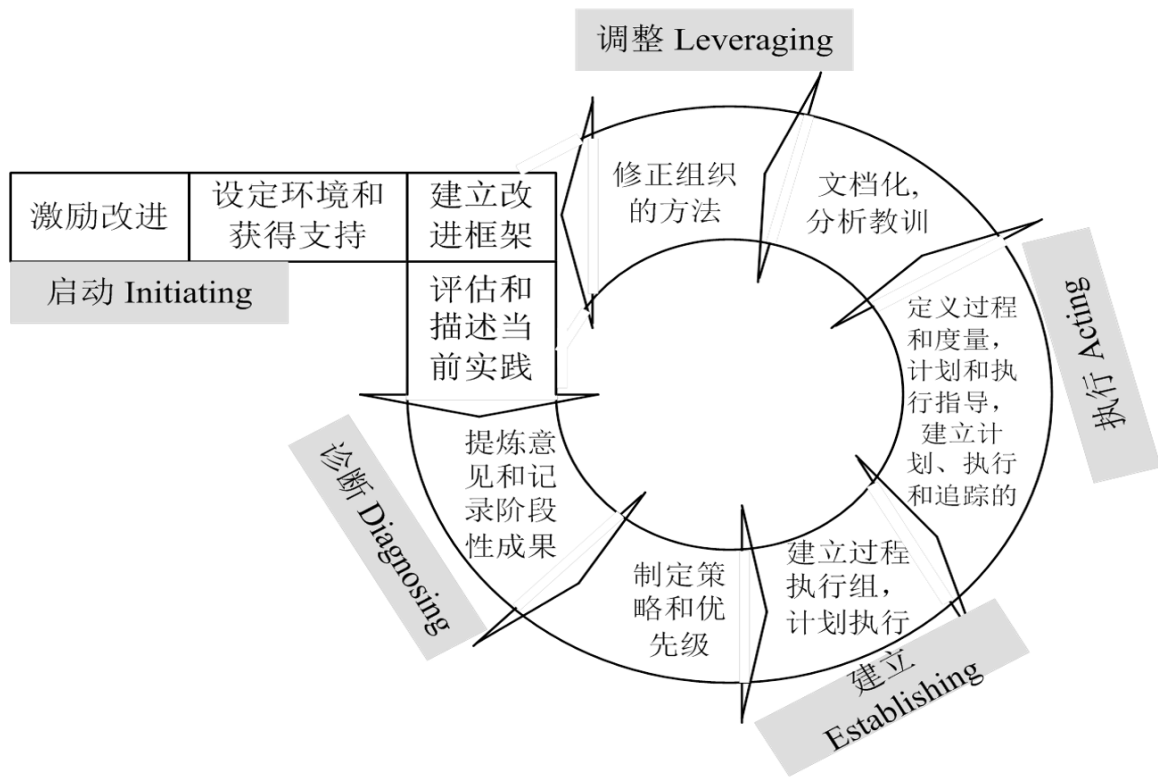
1. 分析现状，找出问题
2. 分析影响质量的原因
3. 找出措施
4. 拟定措施计划
5. 执行措施，执行计划
6. 检查效果，发现问题
7. 总结经验，纳入标准
8. 遗留问题转入下期PDCA循环



图 1.20 戴明的 PDCA 循环示意图

3.4. IDEAL：软件过程改进模型

1. IDEAL模型解决了软件在各种质量改进环境下的需要。
2. IDEAL包括了软件改进过程的五个阶段
 1. Initiating 初始
 2. Diagnosing 诊断
 3. Establishing 建立
 4. Acting 执行
 5. Leveraging 调整



3.5. 其他模型

1. SPICE模型：软件过程管理模型
2. ISO/IEC：软件过程改进模型
3. RUP Rational Unified Process：软件过程框架
 1. 最佳实践：迭代式开发、管理需求、使用基于构建的体系结构、可视化建模、验证软件质量、控制软件变更。
 2. RUP软件开发生命周期：初始阶段、精化阶段、构建阶段、移交阶段。

3.6. 模型理解

1. CMM/CMMI不适合于软件开发的原因：
 1. CMM/CMMI并不是具体的软件过程或者软件开发方法
 2. CMM/CMMI不能作为检验软件过程优劣的标准
 3. CMM/CMMI与其他软件过程或者软件开发方法的比较没有任何意义
2. 判断：
 1. CMMI模型需要适当裁剪以适应公司的实际情况：错误。需要裁减的是公司内部定义的组织级开发流程和开发规范。
 2. CMMI模型太重了，不适合互联网时代的轻量级开发：错误。CMMI根本就不是软件开发模型。
 3. CMMI只适合大公司、大项目，不适合小项目：错误
 4. CMMI只适合需求不变或者很少变化的场合，不适合需求不确定、变化很多的场景：错误。
 5. CMMI不是过程优劣的标准，也不适合用作公司之间的能力比较：对的。CMMI衡量的相对水平。

4. 软件质量管理发展：软件质量大师的主要观点和贡献、工作

1. Shewhart:

1. 最早将统计控制思想引入质量管理，是质量改进奠基人。
2. 提出PDS(Plan-Do-See)模型，后被戴明发展为PDCA。

2. Deming: 日本质量管理之父

1. 提出质量改进的思想
2. 提出PDCA循环(又称戴明环)
3. 提出14条原则: 停止依赖检验的方法获得质量、岗位培训制度化、取消对操作人员规定的工作定额和指标等。

3. Juran

1. 主编质量控制手册，该手册被称为当今世界质量控制科学的"圣经"，奠定了"全面质量管理(TQM, Total Quality Management)"的理论基础。
2. 提出适用性质量: 质量是一种适用性，需要产品在使用期间能够满足使用者的要求。
3. 提出质量三部曲: 质量计划、质量控制、质量改进
4. 提出Juran质量循环
5. 提出80/20循环: 80%的质量问题由领导责任引起。

4. Crosby:

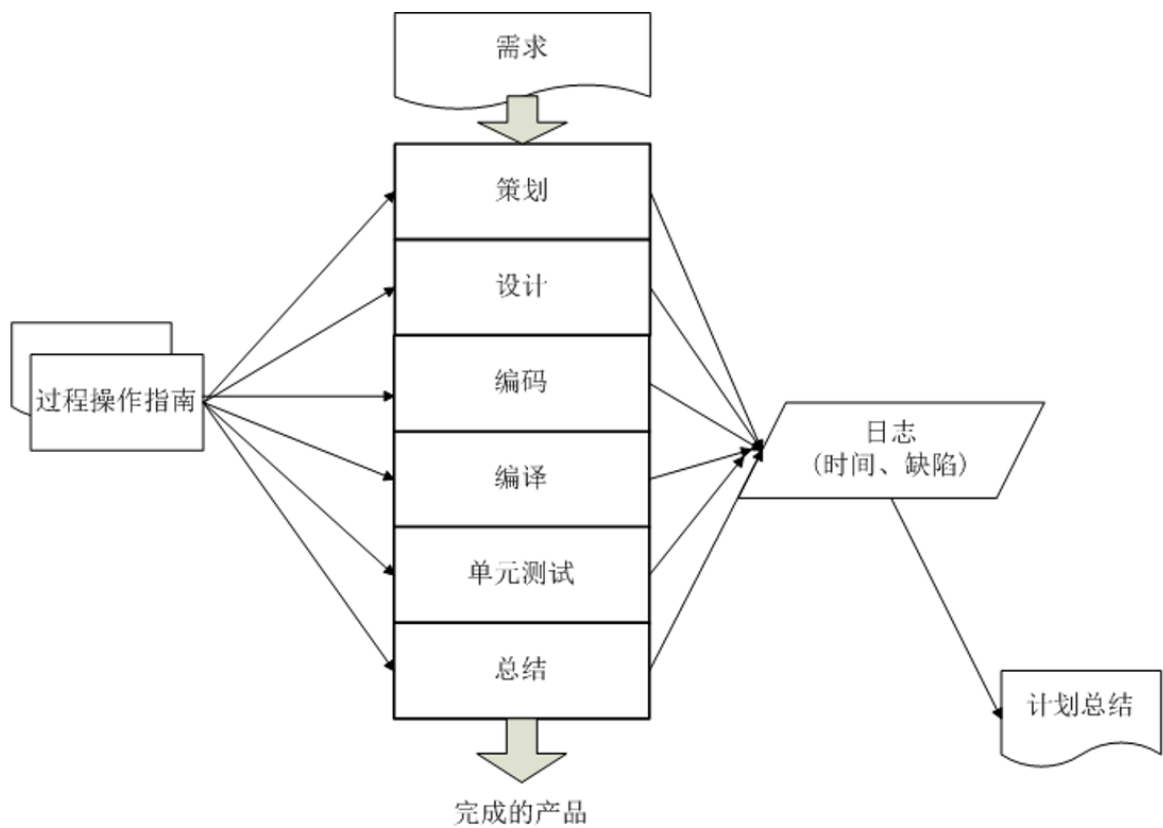
1. 提出了"零缺陷"概念: 将工作放在预防上，而不是质量检验，将事情一次做对。
2. 提出质量管理的绝对性
3. 提出质量改进的基本要素(6C)
4. 发展质量成熟度的度量。

5. Humphrey: 软件过程之父

1. 采用Crosby的成熟度量，提出了软件能力成熟度模型(CMM)
2. 将上述的理论和实践引入软件过程

5. PSP个体软件过程

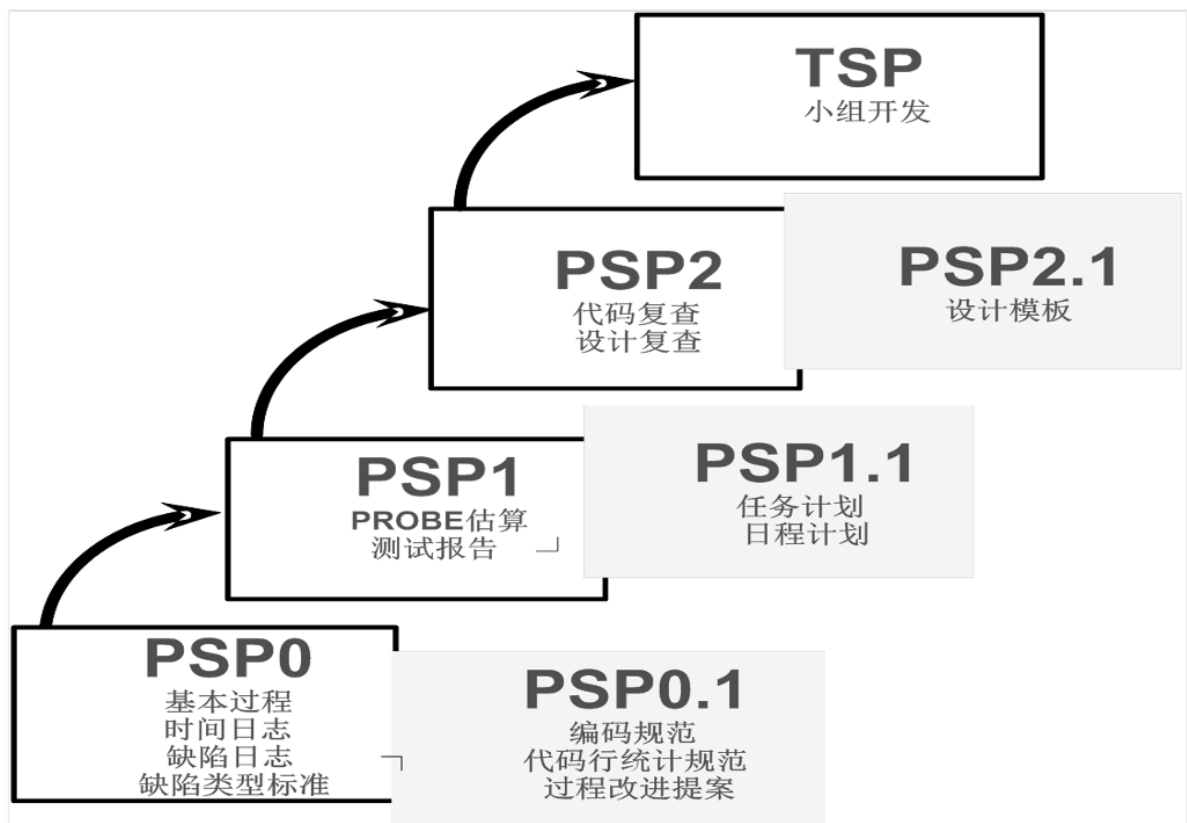
- PSP是一种个体级用于管理和改进软件工程师个人工作方式的持续改进过程。是包括数据记录表格、过程操作指南和规程在内的结构化框架。



5.1. PSP基本原理(为什么有PSP)

1. 软件系统的整体质量由该系统中质量最差的组件所决定。
2. 软件组件的质量取决于开发工程师所使用的开发过程。
3. 软件工程应当自己度量、跟踪自己的工作流，并建立持续的自我改进机制，自己管理软件组件的质量。

5.2. PSP成熟度级别



5.3. PSP质量管理

1. 质量管理是对质量的管理，必须包含管理三要素。
2. 质量实践包括测试等等

5.3.1. PSP的质量观与质量策略

1. 软件项目的日程、成本及质量三大目标统一于质量目标
2. 软件质量是与软件产品满足规定和隐含的需求能力有关的特征或者特征的全体。
 1. 软件质量的内外两部分的特性
 1. 内部质量特性：不直接面向用户
 2. 外部质量特性：面向软件产品的最终用户
 2. 软件质量的不同视角
 1. 软件质量为软件产品可以改变世界，使世界更加美好的程度(用户满意度是最为重要的判断标准)
 2. 软件质量是对人的价值，强调质量的主观性(对同一款软件而言，不同的用户对其质量有不同的体验)
3. 面向用户的质量观
 1. 定义需求为满足用户需求的程度
 2. 这个定义中需要进一步明确：用户究竟是谁？用户需求的优先级是什么？用户的优先级对软件产品的开发过程产生什么影响？怎么度量这种质量观下的质量水平？
 3. 指导意义：开发在前，运维在后；高质量开发确保DevOps中的价值顺畅流动；个体软件工程师的技能、过程直接影响产品质量；PSP关注提升个体软件工程师工程技能
4. 质量策略：
 1. 使用缺陷管理来代替质量管理
 1. 首先确保没有缺陷，然后再考察其他的质量目标。高质量的产品也就意味着软件产品的各个组件基本无缺陷。
 2. 为什么有效？通过关注每个组件的质量可以避免在测试阶段消除缺陷，减少消除代价，提高生产效率。
 2. 各个组件的高质量是通过**高质量评审**来实现的

5.3.2. 质量路径(8个步骤)

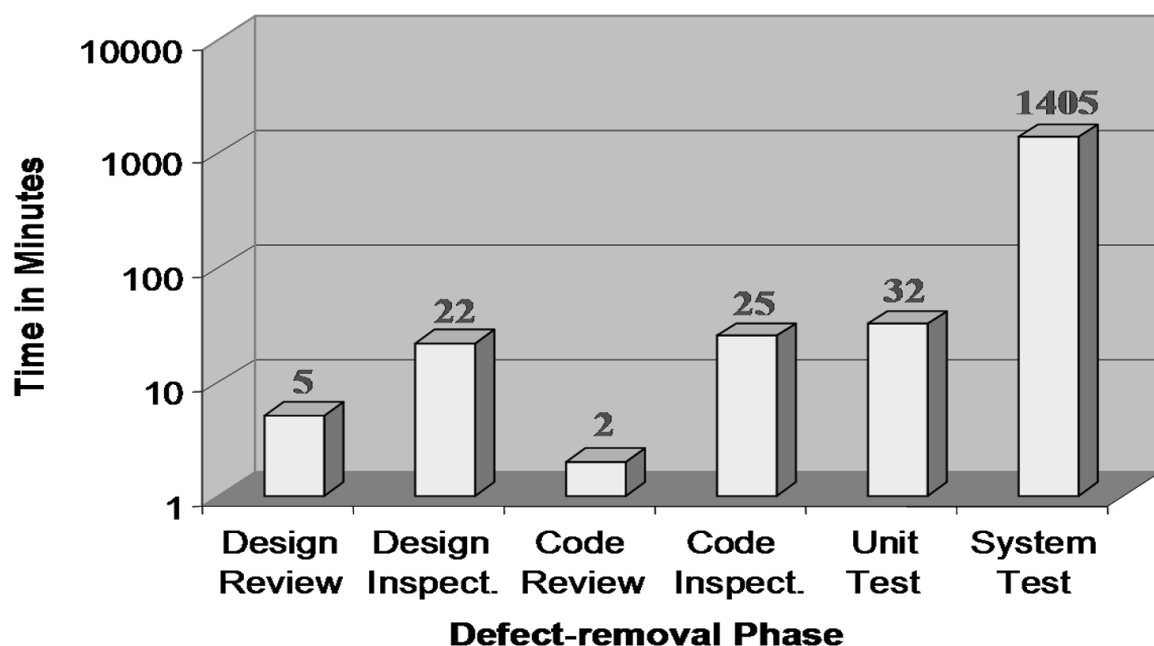
1. 各种测试
2. 进入测试之前的产物质量提升
3. 评审过程度量 and 稳定
4. 质量意识和主人翁态度
5. 个体review的度量和稳定
6. 诉诸设计
7. 缺陷预防
8. 用户质量关 —— 其他质量属性

5.3.3. 评审与测试

1. 测试消除缺陷
 1. 发现待测程序的一个异常行为
 2. 理解程序的工作方式
 3. 调试程序，找到出错的位置，确定出错的原因
 4. 确定修改方案，修改缺陷

5. 回归测试，以确认修改有效
2. 评审消除缺陷
 1. 遵循评审者的逻辑来理解程序流程
 2. 发现缺陷的同时，发现了缺陷的位置和原因
 3. 修正缺陷
3. 评审形式：打印后评审效果更好
 1. 单个屏幕可以展现的内容比较有限：不能完全展示整体结构、安全、性能等
 2. 使用屏幕的评审，评审人员的注意力容易被分散
4. 评审时机
 1. 先编译后评审
 1. 对于某些类型的缺陷而言，通过编译发现并消除缺陷的效率是通过评审发现并消除的数倍。
 2. 一些基于解释执行的集成开发环境，可以消除编译错误。
 2. 先评审后编译
 1. 节省编译阶段的时间：评审的速度和时间是一定的。
 2. 解决编译器遗漏的缺陷。
 3. 编译前评审是良好的学习机会，同时干净的编译给软件工程师带来极大的成就感。
5. 评审的具体形式：先个人评审后小组评审
 1. 小组评审有利于更好的发现缺陷，预防风险，提高Process Yield来确保质量。
 2. 个人评审后安排小组评审，有利于提升个人技能。
6. 评审预防过程中的缺陷数据选择
 1. 选择那些在**系统测试、验收测试以及应用环节出现**的缺陷，特别是验收测试和应用环节中的缺陷，这些缺陷往往意味着软件开发过程本身有不足之处。
 2. 选择那些在**出现频率较高或者消除代价较高**的缺陷，这些缺陷如果可以预防，往往可以节省较多开发的代价，从而体现缺陷预防的优势。
 3. 选择那些**预防方法容易识别和实现**的缺陷，这样的策略容易让软件工程师迅速看到缺陷预防的好处，鉴定使用缺陷预防策略的信心。

缺陷发现的效率：代码评审 > 设计评审 > 设计检查 > 代码检查 > 单元测试 > 系统测试



5.4. PSP度量

5.4.1. 基本度量项

1. 度量时间：序号、所属阶段、开始时间、结束时间、中断时间、净时间、中断原因
2. 度量缺陷：序号、发现日期、注入阶段、消除阶段、消除时间、关联缺陷、缺陷产生原因
3. 度量规模：使用PROBE估算方法
4. 度量日程(TSP)

5.4.1.1. 规模度量标准

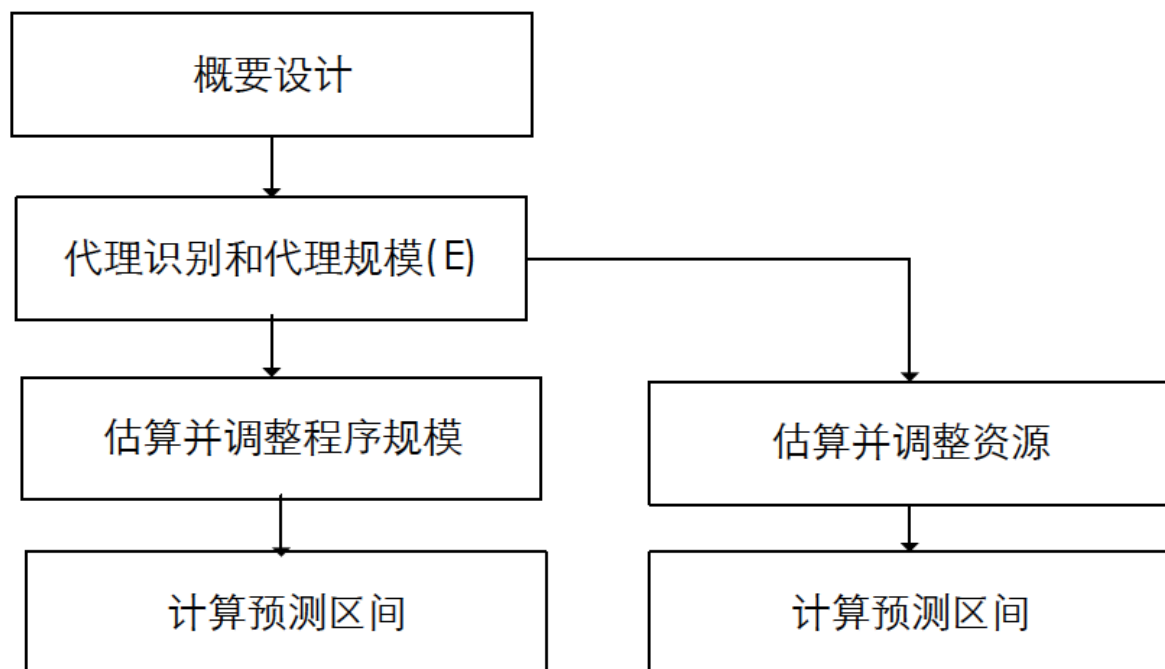
1. 选择的度量方式必须反映开发成本。
2. 选择的度量方式必须精确。
3. 选择的度量方式必须能用自动化方法来统计。
4. 选择的度量方式必须有助于早期规划。

5.4.1.2. 规模度量的困境

1. 精确的度量方式往往不便于早期规划。
2. 有助于早期规划的度量往往难以生成精确度量结果。

5.4.1.3. 解决方案：PROBE方法

1. 估算原理
 1. 设计合理的代理作为精准度量和早期规划所需要的度量之间的桥梁
 2. 使用相对大小而不是绝对大小
2. 方法特点：非常依赖高质量的历史数据，一旦数据不完整或者缺失，则会导致估算结果有偏差。
3. 估算过程：估算是相关干系人达成一致共识的过程。
 1. 概要设计，确认产品功能，明确所需的程序组件/模块，将这些组件/模块与之前缩写的程序比较，估算规模，得到总规模。
 2. 估算要点：尽可能划分详细一些、建立对结果的信息、依赖数据、估算关注的是过程而不是结果。



1. 估算方法：以估算规模为例
 1. 简单方法

1. $VS=\min, S=VS+M2, M=\text{median}, L=M+VL2, VL=\max VS=\min, S=2VS+M, M=\text{median}, L=2M+VL, VL=\max$
2. 计算简单，但是不稳定
2. 正态分布法
 1. $VS=M-2\sigma, S=M-\sigma, M=L=M+\sigma, VL=M+2\sigma VS=M-2\sigma, S=M-\sigma, M=L=M+\sigma, VL=M+2\sigma$ ，均值为MM，标准差为 σ
 2. 相对稳定，在历史数据基本符合正态分布的情况下，可以给出非常好的相对大小矩阵
3. 对数正态分布法
 1. 首先计算所有的数据的自然对数(以e为底)，得到之后的数据的均值MM，标准差 σ ，之后VS、S、M、L、VL计算方法同正态分布法，然后取反对数
 2. 优点：符合人们对程序规模的直观感觉，大多数人习惯写大量规模小的程序，少量规模大的程序
4. 线性回归方法
 1. 项目所需的资源并不是直接由程序规模和历史数据中的生产效率相处得到。为什么？因为程序规模和生产效率都在一个范围内波动，如果直接相除会导致更大误差。
 2. 估算了一定概率条件下的估算值的分布。
 3. Range是一定条件下的变化区间，p为概率
 4. Variance是扰动程度：有时候，历史数据中的一些极端数据会造成相关性假象，因此需要先进行数据降噪

$\text{Plan Size} = \beta_0 \text{ size} + \beta_1 \text{ size}(E)$
 $\text{Plan Time} = \beta_0 \text{ time} + \beta_1 \text{ time}(E)$
 $\text{Range} = t(p, df) \delta_1 + 1/n + (x_k - x_{\text{avg}})^2 / \sum_{i=1}^n (x_i - x_{\text{avg}})^2$
 $\text{Variance} = \delta_2 = 1/n - 2 \sum_{i=1}^n (y_i - \beta_0 - \beta_1 x_i)^2$
 $\text{Plan Size} = \beta_0 \text{ size} + \beta_1 \text{ size}(E)$
 $\text{Plan Time} = \beta_0 \text{ time} + \beta_1 \text{ time}(E)$
 $\text{Range} = t(p, df) \delta_1 + 1/n + 1/\sum_{i=1}^n (x_i - x_{\text{avg}})^2 (x_k - x_{\text{avg}})^2$
 $\text{Variance} = \delta_2 = n - 2 / \sum_{i=1}^n (y_i - \beta_0 - \beta_1 x_i)^2$

1. 处理有限的历史数据

1. 为什么可以使用历史数据完成，使用代理规模与实际程序规模之间的关系完成规模估算？因为规模估算产生偏差的原因相对客观，偏差可以用来修正新的估算结果。
2. 为什么时间估算的偏差产生原因更加复杂：一方面和规模有关，另一方面和人的主观能动性有关。

其中r为相关性(两组数据之间相互关联的程度，PSP中要求 $r \geq 0.7$)，其中S为显著性(两组数据的相关关系出现的偶然性，PSP要求 $s \leq 0.05$)

PROBE 方法	数据要求	数据质量要求	计算方法
A	3 组或者 3 组以上代理规模(E)与实际程序规模。	$r \geq 0.7$; $s \leq 0.05$; $\beta_0 \leq \text{估算结果的 } 25\%$; $0.5 \leq \beta_1 \leq 2$;	略。
B	3 组或者 3 组以上计划程序规模与实际程序规模。	$r \geq 0.7$; $s \leq 0.05$; $\beta_0 \leq \text{估算结果的 } 25\%$; $0.5 \leq \beta_1 \leq 2$;	略。
C	有历史数据	无	按比例调整。
D	没有历史数据	无	猜测。

PROBE 方法	数据要求	数据质量要求	计算方法
A	3 组或者 3 组以上代理规模(E)与实际开发时间。	$r \geq 0.7$; $s \leq 0.05$; β_0 显著小于估算结果; $\beta_1 \leq 0.5 \times (\text{历史生产效率的倒数})$	略。
B	3 组或者 3 组以上计划程序规模与实际开发时间。	$r \geq 0.7$; $s \leq 0.05$; β_0 显著小于估算结果; $\beta_1 \leq 0.5 \times (\text{历史生产效率的倒数})$	略。
C	有历史数据	无	按比例调整。
D	没有历史数据	无	猜测。

5.4.2. 衍生度量项

5.4.2.1. Yield指标

1. 定义了每个阶段在消除缺陷方面的效率。越大越好，希望在80以上。
2. Yield指标是一种事后的质量控制手段，很难非常精确地计算。

$$\text{Phase Yield} = 100 * \frac{\text{某个阶段发现的缺陷个数}}{\text{某个阶段注入的缺陷个数} + \text{进入该阶段前遗留的缺陷个数}}$$

$$\text{Process Yield} = 100 * \frac{\text{第一次编译前发现的缺陷个数}}{\text{第一次编译前注入的缺陷个数}}$$

1. 例子：

阶段	Injected	Removed	remain	Yield
DFD	10	0	10	0
DFD REVIEW	0	4	6	40
CODING	20	2	24	$1/13 * 100$
CODE REVIEW	0	12	12	50
UNIT	0	12	0	100

IBM：最后的测试如果发现了一个错误，那么其中一定还有一个错误没有被发现（所以最后的Yield的值为50）

阶段	Injected	Removed	remain	Yield
DFD	10+4	0	14	0
DFD REVIEW	0	4	10	$2/7 * 100$
CODING	20+8	2	36	$2/19 * 100$
CODE REVIEW	0	12	24	$1/3 * 100$
UNIT	0	12	12	50

1. 基于Yield指标构建缺陷预测模型，并列举该模型的可能改进方案

1. 总体思想：利用回归技术预测软件开发过程中各阶段的Inject Rate(缺陷注入率)和Yield(缺陷消除率)
2. Yield指标只能用来估算，不可以用来度量。结合Yield指标和上图，只需要知道如下指标就可以基于Yield指标构建一个基本的缺陷预测模型：

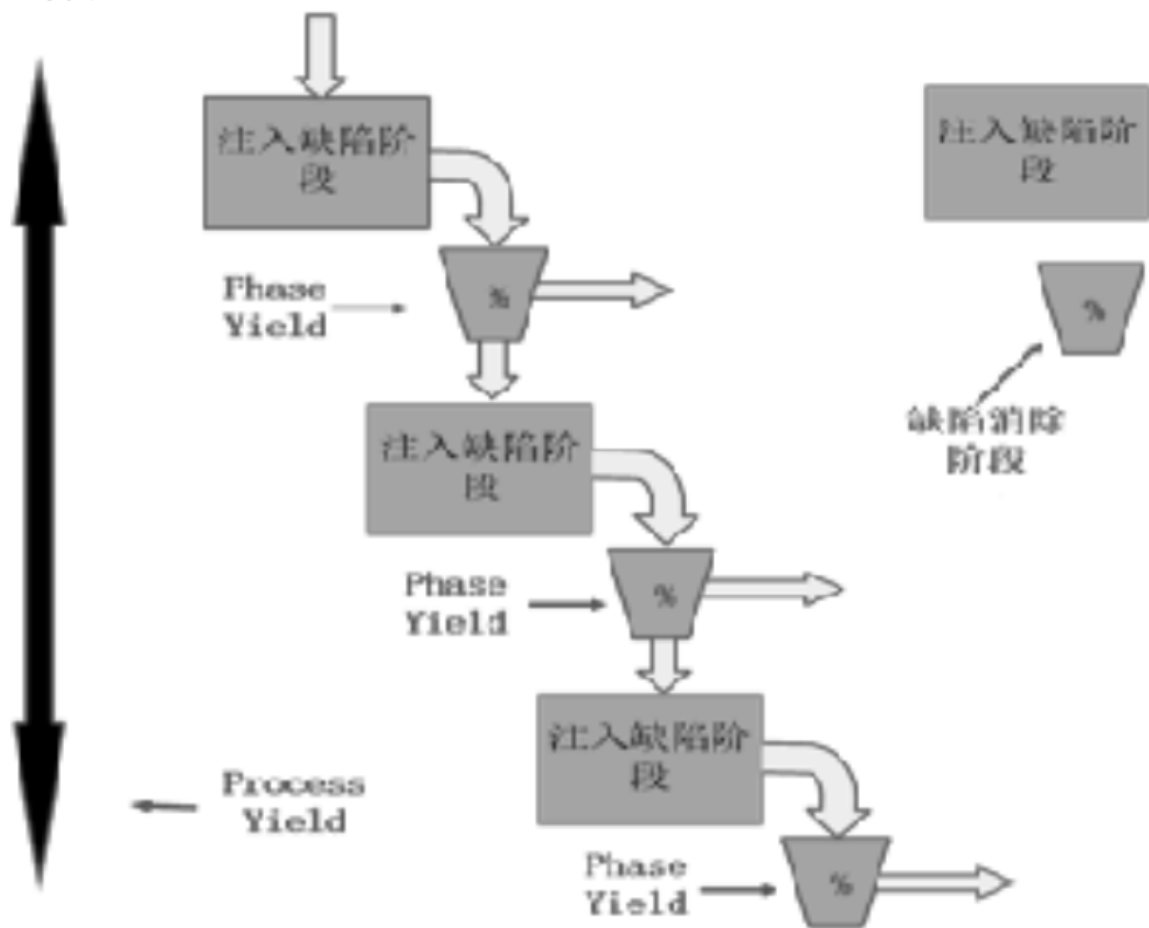
1. 注入阶段注入多少缺陷
2. 缺陷注入的密度（需求每一页注入多少缺陷）
3. 缺陷注入的速度（每小时注入多少缺陷）
4. 消除阶段的缺陷注入密度和速度。
5. 历史数据中的软件规模、文档规模、开发人员规模

3. 步骤

1. 确定纳入影响因子的数据以及数据度量方法
2. 从系统历史库中收集历史数据，并进行整理
3. 依照回归技术进行计算
4. 在项目进行过程中不断收集数据，与预测数据进行比较，调整回归参数
5. 项目过程中依据实际数据与预测数据的误差进行风险的预防、识别和控制

4. 改进方案

1. 维护历史数据：历史数据在简单性、可理解性、稳定性、可度量性、相关性等方面的质量都有影响。
2. 影响因子的选择：不仅仅有软件规模的数据，还需要有开发过程、文档、人员等方面的数据，并将其可度量化。
3. 反馈模型：随着开发实际数据的产生，将这些数据作为输入变量放回模型来调整参数。
4. 蒙特卡洛方法：假设注入水平和消除水平都符合正态分布，计算均值和标准差并用蒙特卡洛方法模拟结果。



5.4.2.2. A/FR

1. COQ(Cost of Quality)

1. 失效成本：分析失效现象、查找原因，做必要的修改所消耗的成本
2. 质检成本：评价软件产品，确定其质量状况所消耗的成本
3. 预防成本：识别缺陷根本原因、采取措施预防其再次发生所消耗的成本

2. 质检失效比

1. $A/FR = \text{PSP质检成本} / \text{PSP失效成本}$
2. 质检成本 = 设计评审时间 + 代码评审时间
3. 失效成本 = 编译时间 + 单元测试时间

3. 理论上，A/FR值越大，意味着质量越高，但A/FR值过大说明评审过多，则开发效率低下，因此PSP中A/FR期望值为2.0

5.4.2.3. PQI

1. 为5个数据的乘积(以基准值作为1，最后结果越接近1，质量越高)

1. 设计质量：设计时间应该大于编码时间， $\min(\text{设计时间}/\text{编码时间}, 1) \min(\text{编码时间}/\text{设计时间}, 1)$
2. 设计评审质量：设计评审的时间应该大于设计时间的50%， $\min(2 * \text{设计评审时间} / \text{设计时间}, 1) \min(2 * \text{设计时间} / \text{设计评审时间}, 1)$
3. 代码评审质量：代码评审时间应该大于编码时间的50%， $\min(2 * \text{代码评审时间} / \text{编码时间}, 1) \min(2 * \text{编码时间} / \text{代码评审时间}, 1)$
4. 代码质量：代码的编译缺陷密度应当小于10个/千行， $\min(20 * \text{编译缺陷密度} / 10, 1) \min(\text{编译缺陷密度} / 10, 1)$
5. 程序质量：代码的单元测试缺陷密度应当小于5个/千行， $\min(10 * \text{单元测试缺陷密度} / 5, 1) \min(\text{单元测试缺陷密度} / 5, 1)$

2. 用途：判断模块开发质量、规划质量活动、过程改进。

5.4.2.4. Review Rate

1. 评审的速度是一个用以指导软件工程师开展有效评审的指标
2. 代码评审速度小于200 LOC(代码行)/h
3. 文档评审速度小于4 page(文档采用页)/h

###j 5.4.3.2. DRL Defect-Removal-Leverage 缺陷消除效率比

1. 缺陷消除效率比：不同缺陷消除手段消除缺陷的效率
2. 计算方法：以**某个测试阶段(一般为单元测试)每小时发现的缺陷数为基础，其他阶段每小时发现缺陷数与该测试阶段每小时发现的缺陷的比值就是DRL**

5.4.2.5. 度量的作用

1. 度量体现着决策者对试图要实现的目标的关切程度。
2. 度量帮助过程的实践者了解过程状态，理解过程偏差。

5.4.3. 考试题

对比度量方式 LOC 和 FP

1. LOC

1. LOC优点：LOC是软件开发项目的生成品，容易进行计算
2. LOC缺点
 1. LOC测量依赖于程序设计语言，不同语言产生结果存在偏差
 2. 对代码量少但设计精巧的程序产生不利评判
 3. 不适合非过程语言
 4. 在软件项目开发前或开发初期估算代码行数非常困难

2. FP

1. FP优点
 1. 和程序设计语言无关
 2. 面向过程和非过程语言均适合
 3. 基于项目开发初期就可能得到的数据
2. FP缺点：计算基于主观而非客观数据
3. 随着软件工程产业化分工加剧专门从事软件下游基础业务的商业组织增多，使得基于LOC的度量方式逐渐失去意义，而基于FP的度量方式显得更加符合要求

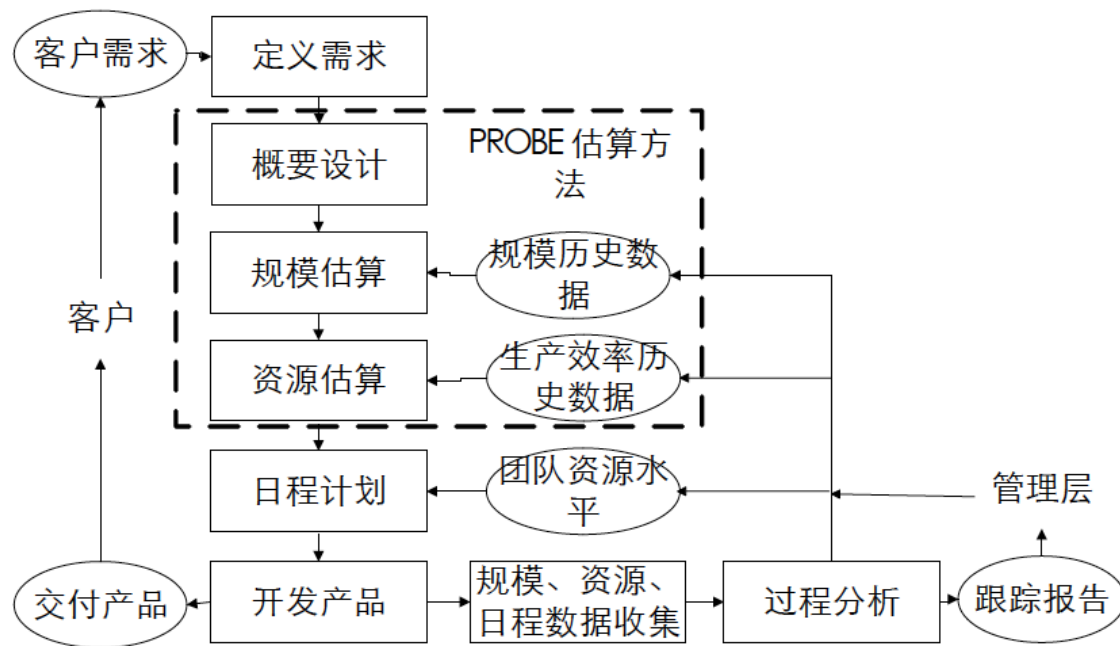
请结合A/FR、PQI、Review Rate、DRL、Yield尽可能具体描述一个软件项目应该如何从多方面来确保开发的高质量（本题满分10分）

这些指标既是开发过程中质量管理的一些参考指标，同时也体现在计划安排中应该注意的质量元素。具体如下：

1. 在项目计划过程中应该安排确保高质量开发结果的活动，例如，按照A/FR、PQI等指标的要求，安排对各类产物（文档和代码）的个人评审和小组评审；
2. 这些评审活动应该满足一定的要求，特别体现在时间方面。例如，评审时间应该多于测试时间的两倍以上（A/FR）；评审时间应该是相应开放时间的50%以上（PQI）；评审速度要求（Review Rate）等
3. 充分借鉴质量指标所体现的开发质量状况，尽早制订相应的质量补救措施。例如，PQI所体现的缺陷密度、所有上述指标的参考值等等。一旦超标，往往意味着质量方面有偏差，应当及时补救。
4. 利用yield等指标，构建质量预测模型，更加积极（Proactive）地判定和控制开发质量；

5. 依据PQI和Yield指标所体现的信息，通过过程改进来提升开发质量。

5.5. 通用计划框架



1. 上述框架中，那些步骤必须人为的干预

1. 定义需求
2. 概要设计：划分由人为开始，规模划分好之后估算是自动产生的
3. 日程计划

2. 这会带来什么的好处？比较容易扛住别人的质疑。

1. 攻击点：资源和时间是否被高估了
2. 解决：估算没有代码行PROBE只有功能点是大中小。

3. 考试题：【2020-mid】请简要描述按照通用计划框架，为了开发合理的项目计划，应该要做哪些估算？PROBE方法充当什么角色。

1. 估算框架如上图
2. 虚线框即为PROBE，用来完成规模和资源的评估

5.6. 个体软件过程的设计

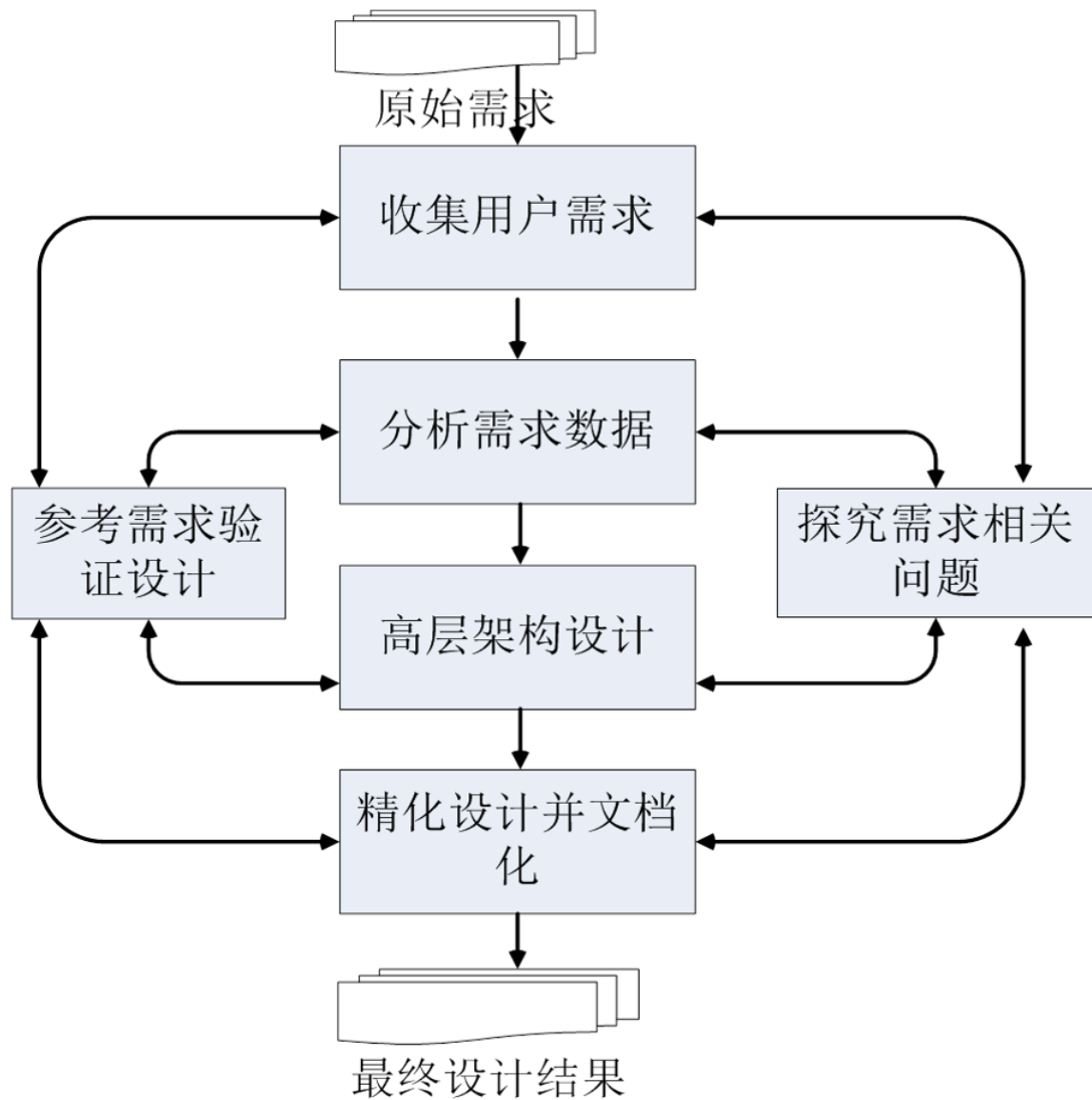
1. 设计 vs. 质量

1. 低劣的设计是导致在软件开发中返工、不易维护以及用户不满的主要原因。
2. 充分设计可以显著减少最终程序的规模，提升质量
3. 设计本身也是一种排除错误的过程。

2. 设计内容：设计目标程序

1. 在整个系统内的位置
2. 使用的方式
3. 与其他组件以及模块之间的关系
4. 外部可见的变量和方法
5. 内部运作机制
6. 内部静态逻辑

3. 设计的过程

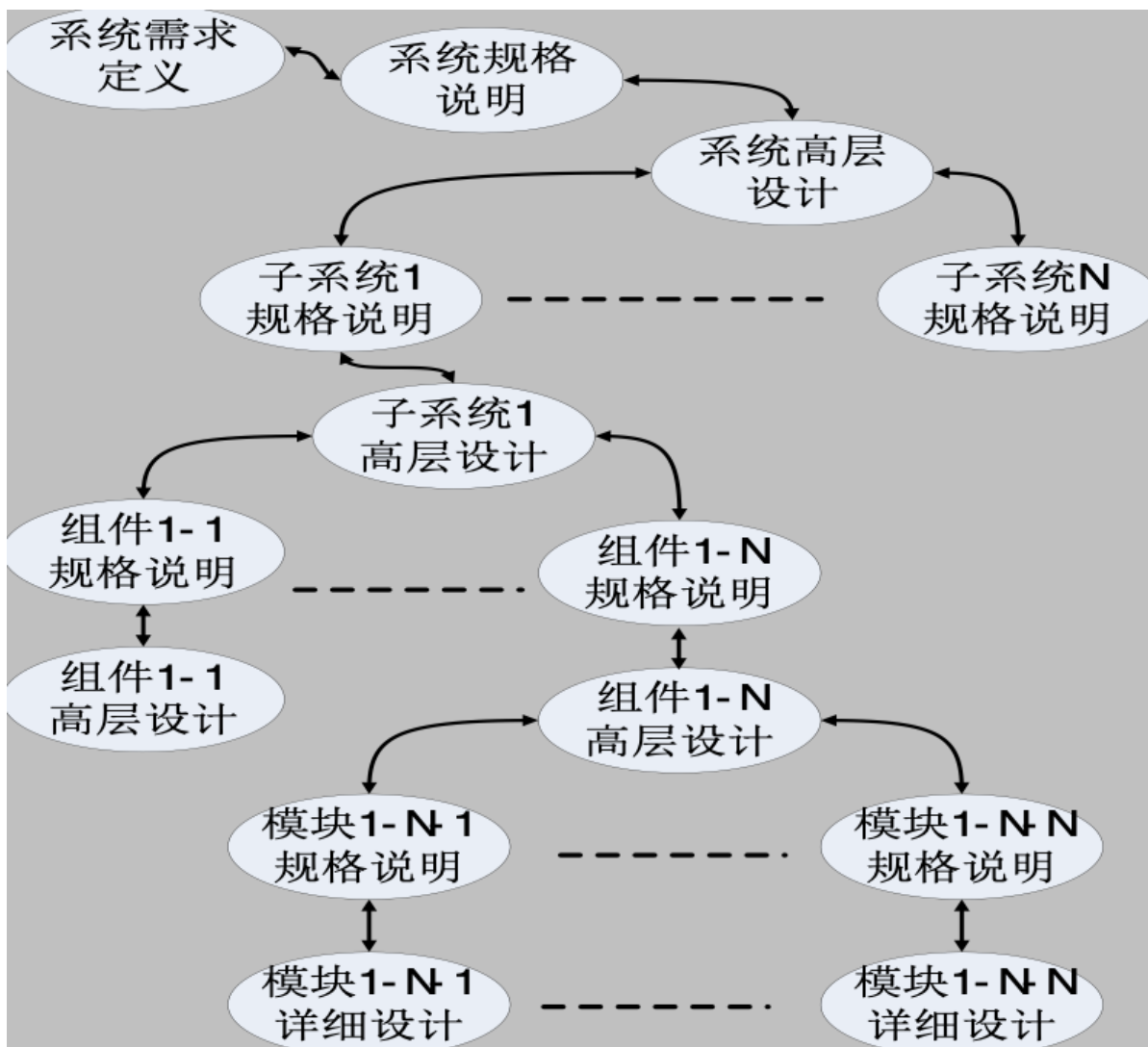


1. 四种设计模块

1. 操作规格模板(Operation Specification Template): 描述系统与外界的交互, 定义测试场景和测试用例。
2. 功能规格模板(Functional Specification Template): 描述系统的对外接口, 用形式化符号描述方法行为, 消除二义性。
3. 状态规格模板(State Specification Template): 精确定义程序的所有状态、状态之间的转换以及伴随每次状态转换的动作。
4. 逻辑规格模板(Logical Specification Template): 精确描述系统的内部静态逻辑, 使用伪代码配合形式化符号来消除二义性。

	动态信息	静态信息
外部信息	交互信息(服务、消息等), OST/FST	功能(继承、类结构等), FST
内部信息	行为信息(状态机), SST	结构信息(属性、业务逻辑等), LST

1. 不同的设计层次



1. 设计验证方法

1. 状态机验证

1. 检查状态机，消除死循环和陷阱状态：使用状态机
2. 检查状态转换，验证完整性(任何一个状态+任何一个条件的组合的下一个状态转换都有定义)和正交性(状态机中任何一个状态的下一个状态的转换条件都不能相同)：使用真值表
3. 评价状态机，检验是否体现设计意图
2. 符号化验证：实施简单，适合不复杂的算法(遗漏系统的改造)；不适合复杂逻辑的场合，纯手工验证方法容易引入错误。
 1. 识别为伪码程序中的关键变量
 2. 将变量使用代数符号表示，重写伪码程序
 3. 分析伪码程序的行为
3. 执行表检查：实施简单，结果可靠，可用于复杂逻辑的验证；每次只能验证一个用例，手工验证比较耗时，容易引入错误。
 1. 识别伪码程序的关键变量
 2. 构建表格，表格左侧填入主要程序步骤，右侧填入关键变量
 3. 初始化被选定的变量
 4. 跟踪被选择的关键变量的变化情况，从而判断程序行为
4. 跟踪表验证：通过符号化、用力识别等方法对程序的一般化行为进行验证，是执行表验证的补充，可以每次验证多个用例。
 1. 识别伪码程序的关键变量
 2. 构建表格，表格左侧填入主要程序步骤，右侧填入关键变量

3. 初始化被选定的变量
4. 识别将伪码程序符号化的机会，并加以符号化
5. 定义并且优化用例组合
6. 跟踪被选择的关键变量的变化情况，从而判断程序行动
5. 正确性验证：将伪码程序作为数学定理，采用形式化方法加以推理和验证。
 1. 分析和识别用例
 2. 对于复杂伪码程序的结构，应用正确性验证的标准问题逐项加以验证
 3. 对于不能明确判断的复杂程序结果，使用跟踪表等辅助验证

6. TSP 团队软件过程

6.1. 需求开发

1. 需求分类
 1. 客户需求：描述客户的期望，使用户解决问题的期望。
 2. 产品需求：描述开发团队所提供的解决方案。
 3. 产品组件需求：描述组成产品的各个组件的需求规格。
2. 为什么"需求是一切工程活动的基础"
 1. 设计活动一定是依据需求而开展的。
 2. V&V活动也是围绕需求展开的
 1. 验证(Verification)是检验获得的产品和产品组件能不能满足事先已经定义好的需求规格。
 2. 确认(Validation)是为了确保产品可以满足客户的需求以及实际操作场景的要求。
 3. 需求也是项目计划活动的关键输入。
3. 需求开发的活动
 1. 需求获取：采用"诱导"式获取客户的显式需求和隐形需求，尽可能的识别客户的期望与所受到的限制。
 2. 需求汇总：整理各种来源的信息，识别缺失的信息，解决冲突的需求(需求的整理和转化、推导未显式描述的需求内容)
 3. 需求验证：对需求进行分析和确认，以确保符合使用者预期
4. 需求文档制作
 1. 完成标志：形成完整的、规范的、经过评审的需求规格说明书。
 2. 意义：使得用户和软件开发双方对该软件的初始规定有一个共同的理解，使之成为整个开发工作的基础。
 3. 优秀文档的特征：内聚、完整、一致、原子、可跟踪、非过期性、可行性、非二义性、强制性、可验证性

6.2. 团队设计

1. 发挥团队智慧的两大挑战
 1. 确定整体架构之前很难进行分工
 2. 鼓励团队成员在讨论和评审会议中的参与程度
2. 设计标准：命名规范、接口规范、系统出错信息、设计表示标准
3. 复用性支持：复用接口标准、复用文档标准、复用质量保证机制
4. 可测试性支持：尽可能减少测试代码的数量、制作合理的测试计划
5. 可用性考虑：

1. 可用性问题需要在设计阶段就开始考虑(而不能推迟到实现阶段)
 2. 针对每一个关键功能都定义操作概念和操作场景
 3. 分析操作场景以确保软件系统开发完成后, 系统使用者会满意
 4. 必要时, 可以邀请最终用户参与场景的评审
6. 设计的文档化

6.3. 实现策略

1. 评审的考虑:
 1. 设计时采用自顶向下、逐层精化的方法, 有利于建立系统的整体观。
 2. 实现时采用自底向上的方法, 方便底层模块的评审和复用。
2. 复用策略
 1. 自底向上的开发策略。
 2. 编码注释的应用: 统一规格, 标明功能
 3. 站立式会议:
3. 可测试性考虑: 实现计划必须与测试计划一致

6.4. 集成策略选择

1. 大爆炸集成策略: 将所有已经完成的组件放到一起进行一次集成
 1. 优点: 测试用例很少。
 2. 缺点: 需要所有有待集成的组件质量比较高, 否则难以定位缺陷位置, 从而消耗很多测试时间; 系统越复杂, 规模越大, 问题越突出。
2. 逐一添加集成策略: 一次添加一个组件的方式进行集成。
 1. 优点: 容易定义缺陷位置(特别在产品组件质量不高的情况下), 每次集成前都有坚实的质量基础。
 2. 缺点: 需要非常多的测试用例; 存在大量的回归测试, 测试时间成本大。
3. 集簇集成策略: 对逐一添加集成策略的改进, 把有相似功能或者有关联的模块优先记性集成, 形成可以工作的组件, 然后以组件为单位进行更高层次的集成
 1. 优点: 可以尽早获得可以工作的组件, 有利于其他组件测试工作的开展。
 2. 缺点: 过于关注个别组件, 而缺乏系统的整体观, 不能尽早发现系统层面的缺陷。
4. 扁平化集成策略: 优先集成高层的部件, 然后逐渐将各个组件、模块的真正实现加入系统, 即尽快构建一个可以工作的扁平化系统。
 1. 优点: 可以尽早发现系统层面的缺陷。
 2. 缺点: 需要编写大量的桩程序, 往往难以覆盖系统应该处理的多种状态。

6.5. 验证与确认

两种活动都是为了提升最终产品的质量而采取的措施

1. 验证(Verification): 检验获得的产品和产品组件能不能满足事先定义好的需求规格。
2. 确认(Validation): 确保产品可以满足客户的需求以及实际操作场景的要求。
3. V&V的区别与联系
 1. 验证和确认的目的不同:
 1. 验证的目的是确保工作产品与事先指定给该工作产品的需求一致。
 2. 确认的目的是确保开发完成的产品或者产品组件在即将使用该产品或者产品组件的环境中工作正确, 关注的是客户需求的满足。
 2. 验证和确认又是互相依存、关系紧密的两个活动

- 1. 验证活动的依据来源于确认的目标，即产品组件需求必须与客户需求一致。
- 2. 验证活动为确认活动提供前提条件，在完成产品需求和产品组件需求之前，考虑客户需求是否满足是没有意义的。

4. V&V的活动

- 1. 单元测试：验证
- 2. 集成：验证
- 3. 需求评审：确认
- 4. 验收测试：确认

	验证	确认
环境准备	评审则准备文件材料等，测试则准备模拟器等	模拟真实环境和场景
对象选择	验证对象从工作产品中选择	确认对象从产品中选择
活动实施	一般的评审和测试工作	包括早期对产品需求评审工作和最后的验收测试
结果分析	关注直到验收阶段才被发现的缺陷	

6.6. 工作分解结构 WBS Work Breakdown Structure

- 1. 工作分解结构是以可交付成果为导向对满足项目目标和开发交付产物的项目相关工作进行的分解。
- 2. 将复杂的项目逐步分解为一系列明确定义的工作任务并作为随后计划活动的指导文档。

6.6.1. 作用/特点

- 1. 提供项目范围基线，是范围变更的重要输入。
- 2. 可以展现项目整体观，使得团队成员更加集中注意力实现项目的目标。
- 3. 为开发项目提供一个整体框架，防止遗漏项目的可交付成果。
- 4. 使得项目中各个角色职责更明确，帮助项目团队的建立和获得下项目项目的承诺。
- 5. 为评估和分配任务提供具体的工作报告的定义，工作报告可以分配给项目某个成员或者另一个团队。
- 6. 是进行估算和编制项目日程计划的基础。
- 7. 可以帮助项目团队理解工作内容，分析项目的风险。

6.6.2. 创建工作分解结构的步骤

- 1. 识别和分析可交付成果以及相关工作。
- 2. 确定工作分解结构的结构与编排方法。
- 3. 自上而下逐层细化分解。
- 4. 为工作分解结构组成部分制定和分配标志编码。
- 5. 核实工作分解的程度是必要且充分的。

6.6.3. 好的工作分解结构的检查标准

1. 最底层要素不能重复，即任何一个工作应该在WBS中的一个地方且只应该在WBS中的一个地方出现。
2. 所有要素必须清晰完整定义，即相应的数据词典必须完整定义。
3. 最底层要素必须有定义清晰的责任人，可以支持成本估算和进度安排。
4. 最底层要素是实现目标的充分必要条件，即项目的工作范围得到完整体现。

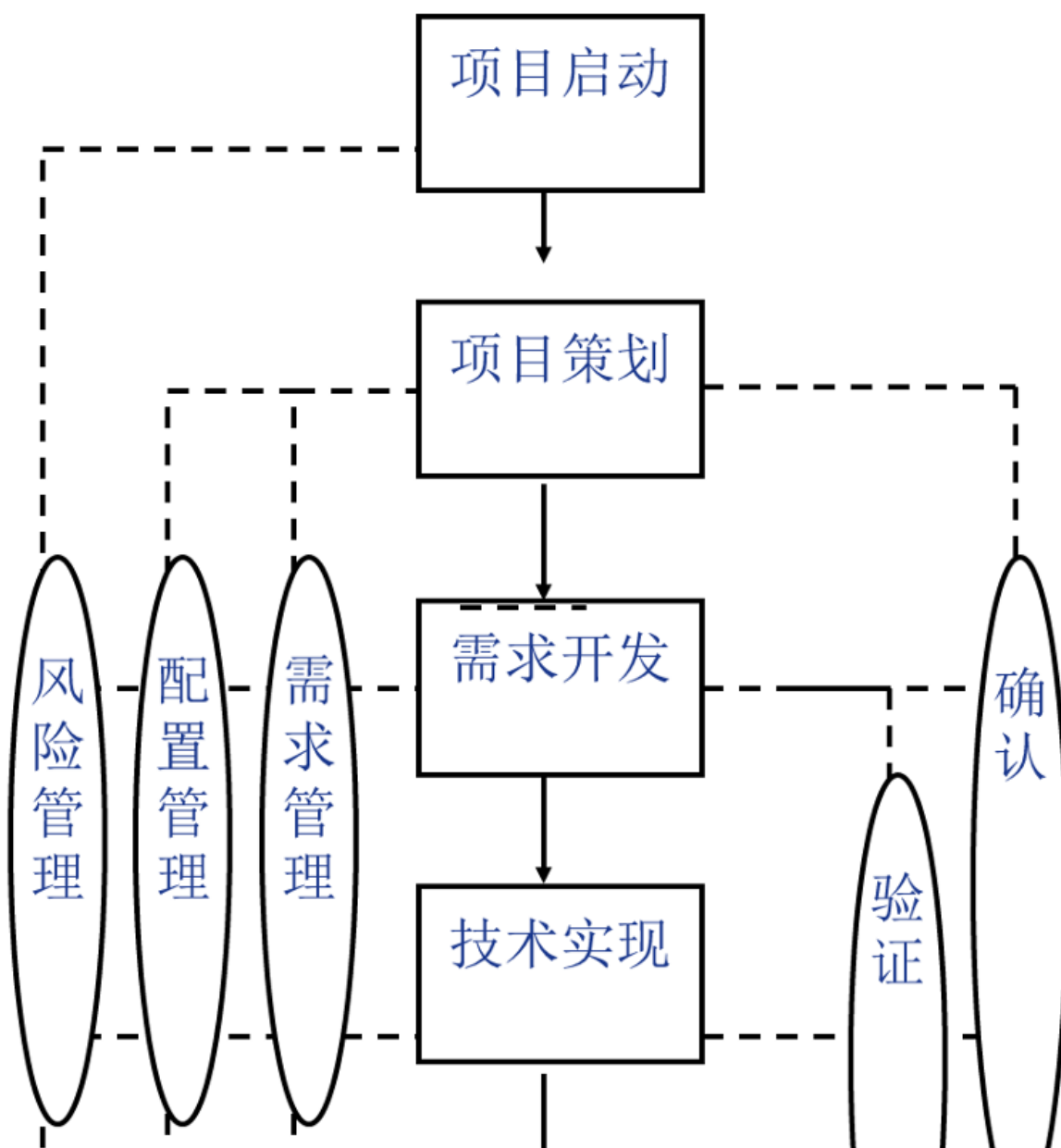
6.6.4. 范围管理

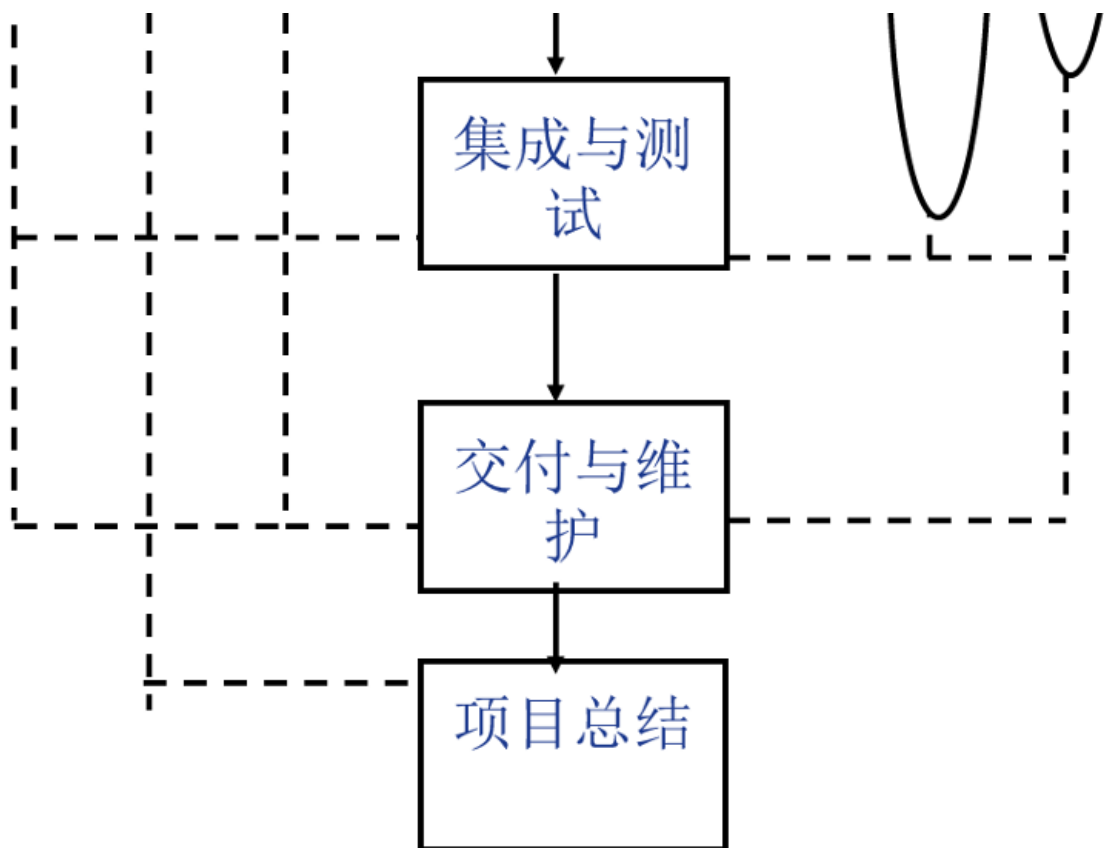
1. 项目范围包括确保项目做且只做完成项目所需的全部工作的各过程
2. 工作分解结构为项目范围管理提供了基础：收集需求、定义范围、创建工作分解结构、核实范围、控制范围变更。

6.7. 开发策略与计划

1. 开发策略与计划是在产品组件需求之上，明确每个产品组件的获得方式与顺序，从而在项目团队内部建立起大家够理解的产品开发策略。

6.8. 生命周期模型选择





6.9. 计划

1. **任务计划**描述了项目所有的任务清单，任务之间的先后顺序以及每个任务所需时间资源。
2. **日程计划**描述了每个任务在日志上的安排，即每个任务计划哪天开始和计划哪天结束。需要考虑资源平衡(所有团队成员几乎同时完成任务)、资源同步。
3. **质量计划**中应当确定需要开展的质量保证活动。
4. **风险计划**
5. 制定资源计划，结合任务计划就可以定义日程计划

6.9.1. 风险管理

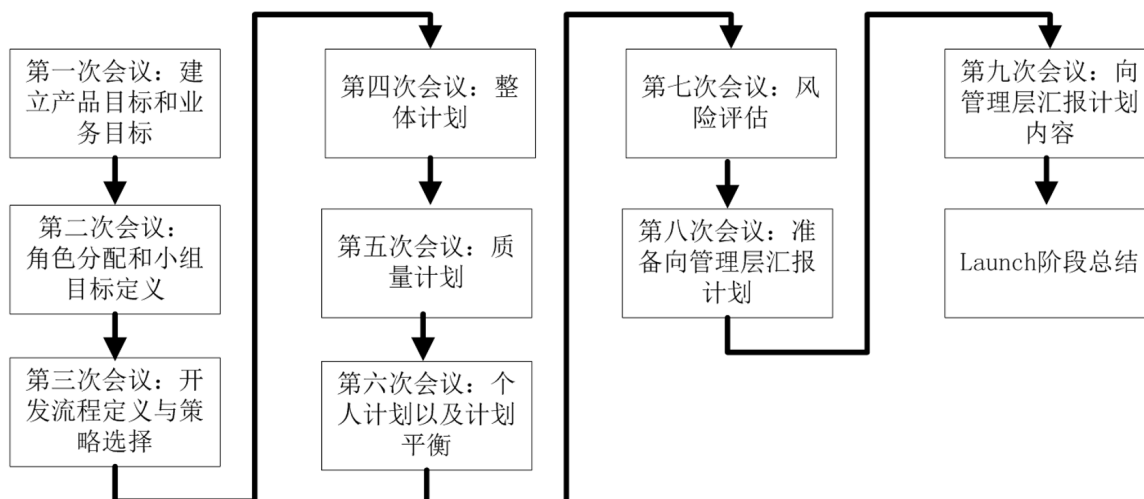
1. 风险管理是一个持续的、前瞻性的过程，需要相关干系人的合作与参与，尽快且积极地识别风险，指定项目风险管理机会。
2. 风险管理的目标是：在风险发生前，识别出潜在的问题，以便在产品或项目的生命周期模型中规划和实施风险管理活动，以消除潜在问题对项目产生的负面影响。
3. 为什么早期风险侦测是重要的？在项目初期进行变更或修改的工作负荷，通常比在项目后期来得更容易、花费较低且不具破坏性。
4. 风险管理包含
 1. 风险识别
 1. 风险：没有发生，有一定概率会发生，发生后有一定影响
 2. 问题：已经发生，比如人员调动等
 3. 典型识别方法：检查WBS的每个组件、使用定义好的风险分类表、访谈相关领域专家、与类似项目进行比较、检查以往项目的总结报告、检查设计规范和需求
 2. 风险应对：识别风险后，就应当制定相应的风险管理策略，以应对各类风险。
 1. 风险转嫁：通过某种安排，在放弃部分利益的同时，将部分的项目风险专家到其他的团队或者组织。
 2. 风险解决：指采用一些有效措施，使得风险的来源不再存在，往往是预防性的手段。

3. 风险缓解：指容忍风险的存在，采用一些措施监控风险，不让风险对项目最终目标的实现造成负面影响。可以通过降低发生可能性/损失程度。

6.9.2. 计划评审与各方承诺

计划完成后，需要与各类计划的相关干系人开展评审工作，达成完整或临时的承诺，由适当层次的人员签署。

6.10. TSP的九次会议



1. 经过平衡的计划和没有平衡的计划有什么不一样？更有把握去成功。
2. 开发和测试都是既有可能引入缺陷，也有可能消除缺陷的阶段。

6.11. 项目追踪

1. 项目追踪的意义：了解项目进度，识别项目实际进展和计划(参照物)的偏差，并且采取适当的纠正措施。

6.11.1. 挣值管理方法 EVM Earned Value Management

1. 挣值管理方法是用来客观度量项目进度的一种项目管理方法。
2. EVM采用进度计划、成本预算和实际成本三个独立的变量，进行项目绩效测量
3. EVM不可以支持质量管理

6.11.1.1. 挣值管理实现

1. 简单实现：仅仅关注进度信息。
 1. 实现方式
 1. 首先需要建立WBS，定义工作范围
 2. 其次为WBS中每一项工作定义一个计划价值（PV, Planned Value）
 3. 最后按照一定的规则将某一数值赋给已经完成的工作或者正在进行的工作，该值成为挣值(EV, Earned Value)
 2. 常用规则
 1. 0-100原则：只有当某项任务完成时，该任务的PV值将转化成EV值
 2. 50-50原则：只需要开始某项任务，即可以赋原PV值的50%作为EV值，完成时，再加上另外的50%
 3. 实际完成的工作所需成本AC不对EV值产生任何影响

2. 中级实现

1. 在简单实现的基础上，加入日程偏差的计算，加入了成本线(AC)

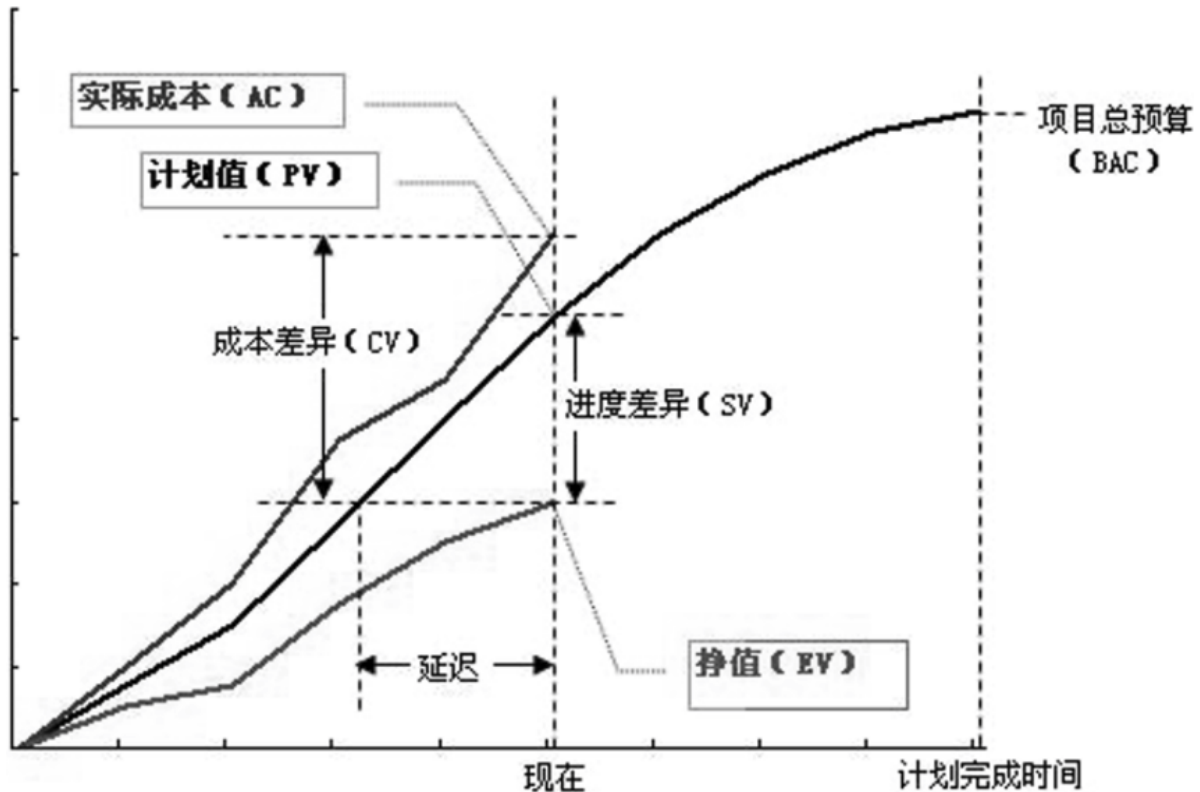
2. 典型计算方式有

1. 日程偏差 $SV = EV - PV$

2. 日程偏差指数 $SPI = EV/PV$;

3. 高级实现：添加预测线（BAC），当任务足够多的时候，我们就可以让预测线尽可能平直，同时我们延伸挣值（EV），找到与预测线（BAC）的交点，我们就可以明确项目的落后时间

6.11.1.2. 挣值管理图解



1. 上面的线是为了获取这些挣值付出的实际代价，这个线和挣值之间的差异是成本差异。

2. 中间的线是预算（每天需要完成多少挣值）BAC，理想情况下是一条直线。

3. 下面的线是挣值（实际的进展情况）（EV），和owner value有关，对应plan value

4. 实际获取挣值和预计获取挣值的差异是进度差异。

5. 例子：

1. 一共100个值（计划投入时间），第一天需要完成50个值，第二天需要完成20个值，第三天需要完成30个值。

2. BAC：第一天50，第二天70，第三天100。

3. 超期情况，EV：第二天50值，AC：第二天100值。

1. 这就意味这两天投入了100个值完成了原计划用50个值完成的工作，这就对应着进度落后，成本超支。

2. $CV = 50$ ， $SV = 20$

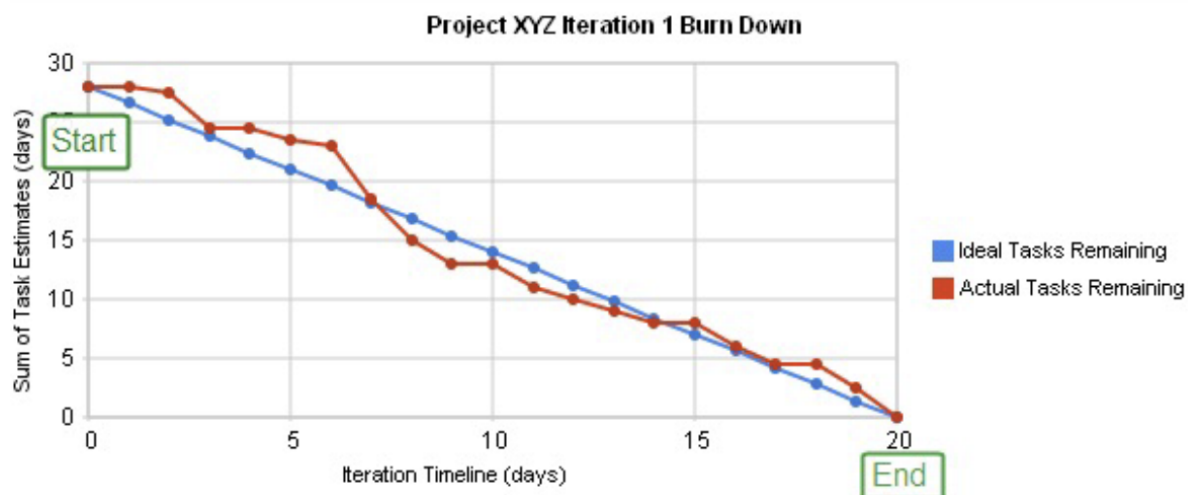
4. 有可能出现第二天用户说，任务B和任务C不做了（项目进度落后变为了项目进度正常）

6. 挣值管理会带来什么好处？可以很好的适应项目的动态变化。

6.11.1.3. 挣值管理的度量

1. BAC表示按照PV值的曲线，当项目完成的时候所需预算或者时间
2. 成本差异 $CV = EV - AC$ ，表示的是已经完成的工作与所消耗的成本的差异。可以表示为消耗的时间，也可以表示为消耗的资金。
3. 成本差异指数 $CPI = EV/AC$ ，表示单位成本创造的价值
 1. $CPI < 1$ 说明成本超支
 2. $CPI = 1$ 说明成本与预期一致
 3. $CPI > 1$ 说明成本低于预期。
4. 日程偏差 $SV = EV - PV$ ，表示进度偏差。
 1. $SV < 0$ 表示进度落后； $SV = 0$ 表示进度正常
 2. $SV > 0$ 表示进度超前。
5. 日程偏差指数 $SPI = EV/PV$
6. 预计完成成本 $EAC = AC + (BAC - EV)/CPI = BAC/CPI$ ，表示的是按照目前的进展已经成本消耗情况，整个项目完成的时候所需消耗的成本。

6.11.2. 燃尽图：挣值管理的变形



1. 燃尽图是简单的挣值管理的变形。
2. 他是剩下的工作占的百分比。

6.11.3. 里程碑评审

1. 软件项目的里程碑往往是某个时间点，用来标记某些工作的完成或者阶段的结束。比如完成某项工作、获得干系人签字认可、完成某产品的评审、修改或交付某产物。
2. 里程碑评审的审查内容：项目相关承诺、各项计划的执行状况、当前的状态讨论、面临的风险讨论。

6.11.4. 纠偏活动管理

1. 偏差原因分析：收集信息，分析信息，找出偏差的根本原因
2. 纠偏措施定义：确认偏差的根本原因，定义纠偏的措施。
3. 纠偏措施管理：管理纠偏措施直到结项。

6.11.5. 项目审查

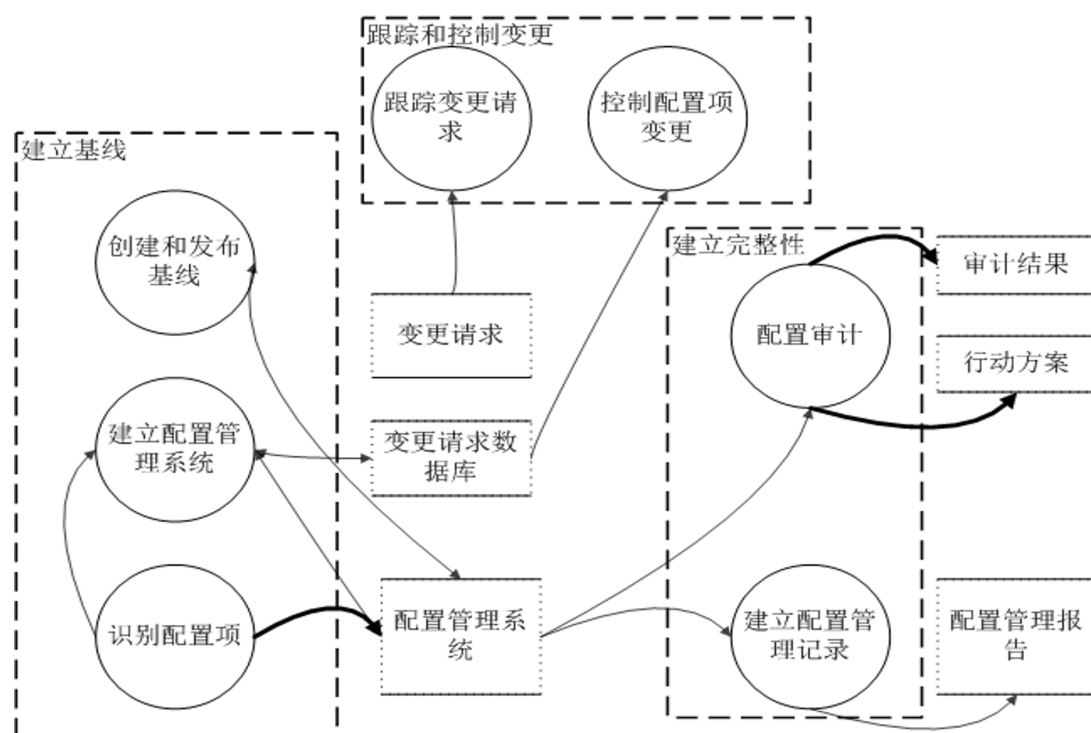
6.12. 项目总结

1. 基于PMBOK的总结方式：包含范围管理、时间管理、成本管理、质量管理、人力资源管理、沟通管理、风险管理、采购管理和整合管理9大知识领域(每个领域具体内容见Lec-4)
2. 基于TSP的项目总结方式
 1. 准备阶段
 2. 过程数据评审阶段：由过程经理或质量经理带领整个团队分析过程数据，识别过程改进机会。
 3. 人员角色评价阶段
 4. 总结报告撰写阶段
3. 意义：提供一个系统化的方式来总结经验教训，防止犯同样的错误、评估项目团队绩效、积累过程数据等，给项目团队成员持续学习和改进的机会。

6.13. 项目管理支持活动

6.13.1. 配置管理

1. 目的：建立与维护工作产品的完整性。
2. 概念
 1. 配置项：在配置管理当中作为单独实体进行管理和控制的工作集合。
 2. 基线：一个或多个配置项及相关的标识符的代表，是一组经正式审查同意的规格或工作产品集合，是未来开发工作或交付的基础，而且只能经由严格的变更控制程序才能改变。
 1. 发布时间点：需求分析后、设计完成后、单元测试后、最终产品发布后
 2. 配置项持续演进的稳定基础
3. 配置管理活动：

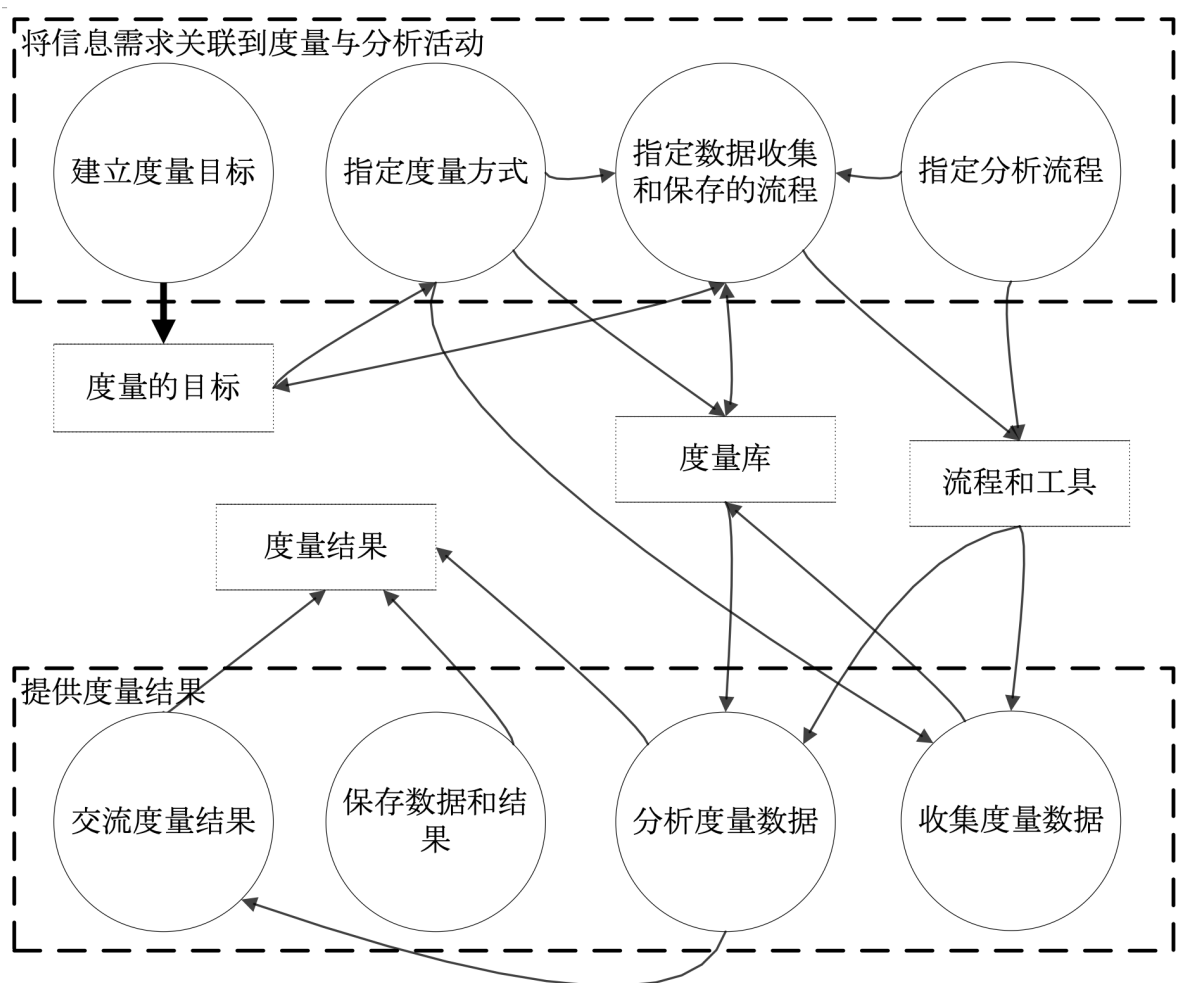


1. 如何控制变更

1. 跟踪变更请求：启动程序、分析影响、审查相关项、跟踪直到结项

6.13.2. 度量和分析

1. 目的：支持管理的信息需要。
2. 度量和分析活动



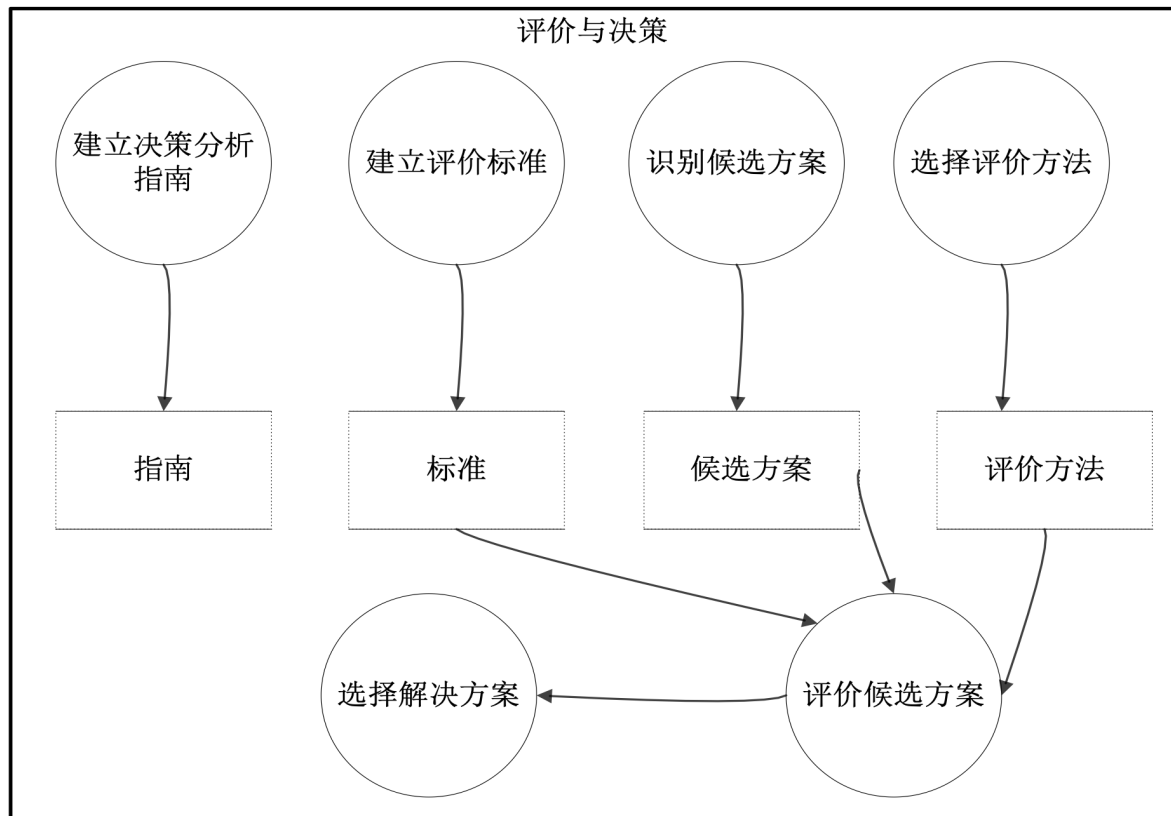
1. 意义：基于客观的数据对决策很重要，可以显著消除错误决策的风险。而客观数据的获取需要按照一定的流程用正确的方式获得和使用。

6.13.3. GQM方法

1. GQM(Goal Question Metric)是一种应用非常广泛的建立软件度量体系的方法，从管理的目标出发，将目标归纳、分解为度量的指标，并把这些指标提炼成可以测量的值，是一种科学的、系统的思考问题的方式。
 1. 概念层(目标)：目标是为某个特定的对象而定义的。这里的对象是指软件产品、软件过程以及相关的资源等。定义的目标基于不同原因和不同质量模型,也要参考不同的角色视图与特定的环境。
 2. 操作层(问题)：基于一定的刻画上述目标是否达成或者目标达成的进展情况的模型，使用一系列的问题来定义所研究的对象, 然后得出评价或评估特定目标达成进展情况。所选择的问题应当尽量体现质量相关的话题。
 3. 量化层(度量)：试图以量化的方式回答上述操作层识别出来的问题。

6.13.4. 决策分析

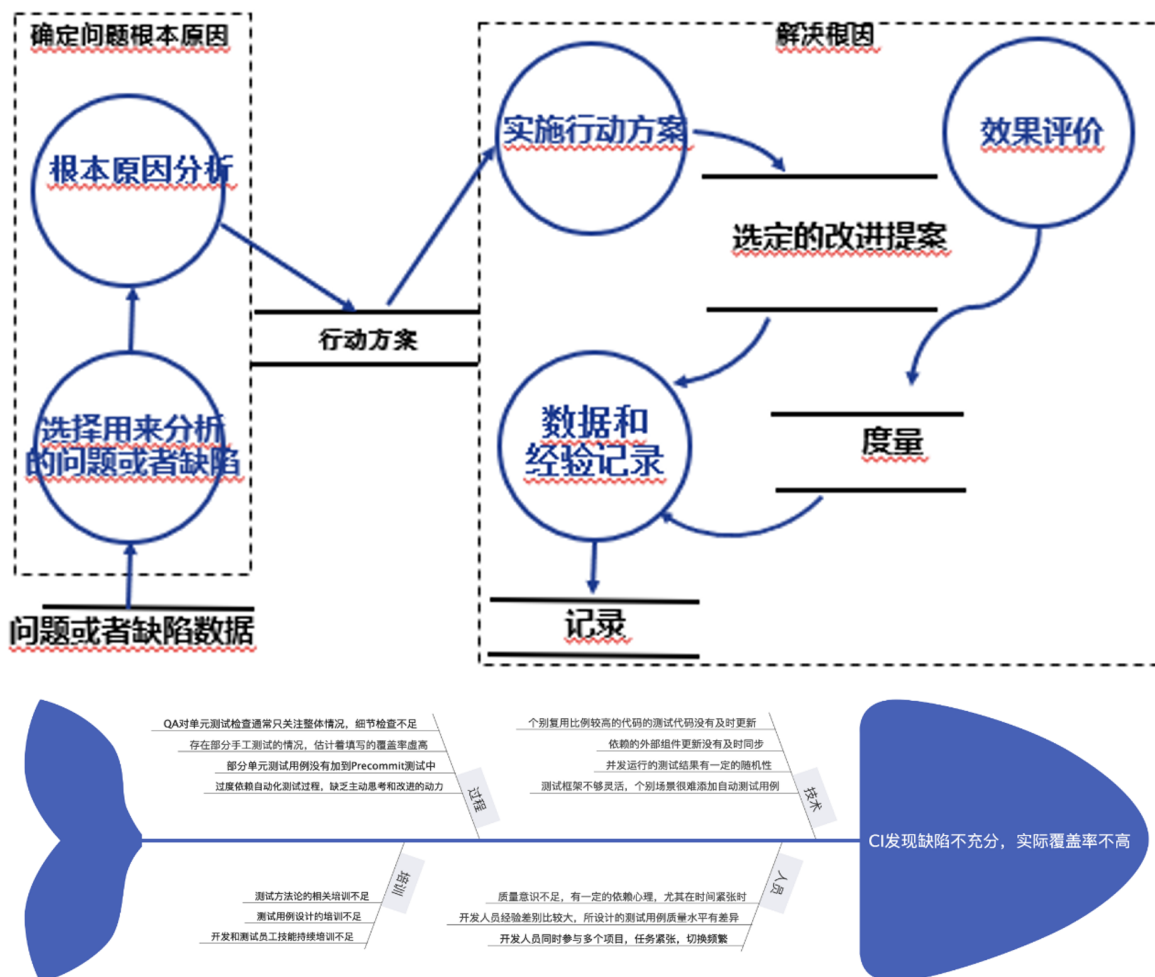
1. 决策分析的活动



1. 决策分析的意义：错误的决策往往会给项目带来灾难性后果。为了降低这种错误决策的风险，往往需要尽可能基于客观事实和正确的流程来开展决策与分析活动

6.13.5. 根因分析

1. 避免类似错误反复发生



6.14. 团队动力学

1. 软件开发特点

- 软件开发是一项既复杂又富有创造性的知识工作。
- 软件开发是智力劳动，需要处理和讨论极其抽象的概念，并将不同的部分整合成一个可以工作的系统。
 - 要求从事软件开发的工程师必须全身心的参与工作、主观意愿上努力追求卓越。
 - 要求管理者激励并维持激励

2. 管理知识工作的关键规则：管理者无法管理工作者，知识工作者必须实现并学会自我管理。

6.14.1. 自主团队

1. 定义：

- 一个团队必须包括至少**两个成员**，他们为了**共同的目标和愿景**而努力工作，他们每个人都有**明确的角色和相应的职责定义**，任务的完成需要团队成员**互相依赖和支持**。
- 如果团队成员都实现了自我管理，也就形成了所谓的自主团队。

2. 特点：

- 自行定义项目的目标
- 自行决定团队组成形式以及成员的角色
- 自行决定项目的开发策略
- 自行决定项目的开发过程
- 自行制定项目的开发计划
- 自行度量、管理和控制项目工作

3. 形成过程：不是偶然形成的，一开始团队成员有不同的目标，经过一段时间的协同工作后，团队成员慢慢适用，演化成自主团队。

4. 必要性

:

1. 自主团队可以形成胶冻状团队
2. 团队成员互相支持，任何时刻都指导应该以怎样的方式帮助别人，互相信任有强烈的归属感。
3. 团队在适当的时候会聚集在一起，研究现状，讨论策略。

5. 外部环境：在项目启动阶段、项目进展过程中获得管理层的支持

6.14.2. 激励方式

1. 威逼、利诱、鼓励承诺(位于马斯洛需求理论的4级以上，最好以团队为单位)

1. 威逼指完全依靠不同角色的等级关系强制要求下属必须完成某些工作，满足生理需求。
2. 利诱是通过许诺一定的好处来吸引下属努力工作，满足生理需求 and 安全感
3. 鼓励承诺是通过建立承诺文化，利用软件工程师希望得到别人尊敬的心理，鼓励他们合理承诺并努力满足承诺，从而获得尊敬。

2. 威逼和利诱属于交易型领导方式，鼓励承诺属于转变性领导方式。

3. 承诺文化建立与团队激励

1. 在满足以下条件下，团队承诺比个人承诺的激励作用更大：所有成员参与、团队依赖于每一位成员履行自己的承诺、有计划支撑。
2. 制定承诺时需要保证：承诺是自愿的、公开的、可信(行)的，向团队做出承诺、有详细计划支撑，开发者参与。
3. 及时提供各种反馈信息是维持激励的有效手段。

4. 需求理论

1. 马斯洛需求理论：胜利需求、安全感、爱和归属、获得尊敬、自我实现
2. 海兹伯格：激励因素(内在因素，成就感等)、保健因素(外在因素，薪水等)
3. 麦克勒格的X理论：人性本恶，独裁式的管理风格。
4. 麦克勒格的Y理论：人性本善，民主式的管理风格。
5. 期望理论： $Motivation = Valence * Expectancy$
 1. M：激发力量，积极性
 2. V：目标价值(效价)，达到目标对于满足个人需要的价值，有正零负三种，效价越高，激励力量越大
 3. E：期望值，人们根据过去经验判断自己达到某种目标的可能性大小

6.15. TSP的典型角色

自行查看

7. 定量管理与仿真建模

1. 一般的项目管理：CMMI的1-3级

1. 主要关心当前状态，是否需要调整干预
2. 但是回答不了进行预测的问题。

2. 高成熟度项目管理：CMMI的4、5级，也就是定量管理。

1. 基于数据，主要关心当前状态理解是否有信心，对偏差的理解
2. 可以回答预测类的问题

3. 为什么需要理解偏差：解决部分问题，比如希望10周以内交付，历史数据表明类似任务是9-11周完成。
4. 定量管理基本范式：
 1. 构建定量模型：子过程能力基线、过程模型
 2. 应用模型：监控影响子过程的关键因素
5. 概念
 1. (子)过程性能：遵循某个特定（子）过程的之后产生结果的量化描述，既包括（子）过程度量Xi（例如，时间、缺陷消除效率、工时等），也包括产物度量Yi（例如，缺陷密度，相应时间等）。
 2. (子)过程性能基线：上述过程性能的一个量化的刻画，一般包括均值和范围。通常用作过程性能的benchmark。
 3. 过程或子过程性能模型：依据子过程的逻辑关系构建相应的数学模型，描述子过程性能基线和整体过程有意义的性能输出（例如，质量、生产效率、成本等）之间关系。例如过程Yield和Phase Yield。
6. 常用定量管理技术
 1. 非统计技术：一般是为了描述数据集整体特征或者关联关系，从而帮助选择适用的统计技术以应用于给定的数据集，以及调查通过数据分析检测到异常的原因。例如，**检查表（或列联表）**，**帕累托图**，**直方图**，**因果图**，**散点图**等。
 2. 统计技术：统计思维认识到许多决策是在**不确定条件**下做出的。统计方法有助于**量化这种不确定性**并指导采取行动以减少不确定性。常用的统计技术包括：**统计过程控制图**，**回归分析**，**方差分析**，**预测区间**，**假设检验**，**敏感性分析**等

8. DevOps

8.1. 精益

1. 精益屋的两大支柱：
 1. 准时化：只在顾客需要的时候，按照顾客所需要的量，生产顾客所需要的产品。
 2. 自动化：将人的智慧赋予机器
2. 精益理论：降低批量规模、减少半成品，缩短并增强反馈回路，DevOps是IT价值流中应用精益理论的结果。
3. 精益原则
 1. 全局优化：局部优化长期而言对系统整体优化不利
 2. 消灭浪费：避免构建错误的功能、拒绝学习、辗转现象
 3. 品质为先：如果验证中总能发现缺陷，那么流程就有问题
 4. 不断学习：规划工作非常有用，学习必不可少
 5. 尽快交付：建立稳定、连贯的工作流
 6. 人人参与
 7. 不断提高

8.2. DevOps

1. Devops：开发运维一体化，是软件开发、运维和质量保证三个部门之间的沟通、协作和集成所采用的

流程、方法和体系

的一个集合。

1. 方法论基础是**敏捷软件开发，精益思想以及Kanban方法**
2. 以**领域驱动设计**为指导的**微服务架构方式**
3. 大量**虚拟化技术**的使用(开发、测试环境)
4. **一切皆服务**XaaS的理念指导
5. 构建了强大的**工具链**，支持水平自动化

2. Devops与敏捷的关系

1. DevOps思想部分起源于一些运维工程师借鉴软件敏捷开发思想
2. DevOps的早期活跃分子来自敏捷社区
3. 公认的适用于DevOps的软件过程方法是敏捷软件开发，尤其是Kanban

3. Devops的

特点

1. 让开发和运维共同为业务的成功负责
2. 开发和运维技能一部分融合

4. Devops化的三个阶段

1. System Thinking
 1. 充分理解工作流(从开发到IT运维再到客户)，完成整体产品
 2. 实践和方法：限制半成品、CI/CD
2. Amplify Feedback Loop
 1. 快速持续反馈，放大其效益，避免问题再次发生，从源头上保证质量。
 2. 实践和方法：持续改进、适当停止生产线
3. Culture Of Continual Experiment and Learning
 1. 培育不断尝试、重复和练习的文化
 2. 实践和方法：营造用于创新、敢于冒险以及高度信任的企业文化。
4. DevOps与敏捷开发

8.3. 软件架构

1. 什么是软件架构

1. 架构是一系列重要决策的集合，包括：软件的组织，构成系统的结构要素和接口选择，元素在协作中的行为
2. 架构是计算组件和组件之间的交互：连接件和约束

2. 微服务架构：将单一应用程序开发为一组

小型服务

的方法。每个服务运行在

自己的进程

中，将应用拆分成多个小业务单元来开发和部署，使用

轻量级协议

通信，通过协同工作实现应用逻辑的架构模式。

1. 微服务的理念：分而治之，本质上也是**分布式架构(面向服务(SOA)的一种扩展)**
2. 特点
 1. 应用程序分解为具有明确定义了职责范围的细粒度组件
 2. 完全独立部署，独立测试，并可复用

3. 使用轻量级通信协议，HTTP和JSON，松耦合
4. 服务实现可使用多种编程语言和技术
5. 将大型团队划分成多个小型开发团队，每个团队只负责他们各自的服务

8.4. 一些概念

1. 云计算是一种按量付费的模式，这种模式可以提供可用的、边界的、按序的网络访问，进入可配置的计算资源共享池(资源包括**网络、服务器、存储、应用软件、服务**)，这些资源能够被快速提供，只需投入很少的管理工作，或服务供应商进行很少的交互。
2. XaaS表示一切皆服务，它是一个统称(anything as a Service或everything as a Service)
 1. 典型的三大XaaS
 1. Software as a Service (SaaS) 软件即服务，软件分发方式，中心化，服务供用户订阅。
 2. Infrastructure as a Service (IaaS) 基础设施及服务，虚拟化，用户需要配置和部署中间件和应用服务。
 3. Platform as a service (PaaS) 平台即服务，服务供应商提供开发的整体环境。
 3. **云原生**是一种基于微服务的软件架构思想以及基于Devops进行软件开发实践的一组方法论
 4. **容器**通常指在一个隔离的环境中运行的进程以及进程所依赖的环境的总称。
 5. CI/CD：持续集成、持续交付、持续部署。
 6. Docker：容器将一个软件封装在一个完整的文件系统中，该文件系统包含它运行所需的一切：代码、运行时、系统工具、系统库
 7. Git：分布式版本控制系统，强调速度、数据完整性和对分布式非线性工作流的支持
 8. Jenkins：持续集成工具，配置流水线等
 9. **运维**主要指软件系统测试交付后的发布和管理工作，其核心目标是将交付的业务软件和硬件基础设施高效合理的整合，转换为可持续提供高质量服务的产品，同时最大限度降低服务运行的成本，保障服务运行的安全

8.5. 几个问题

1. JIT及时生产，价值流和价值拉动的关系
2. 微服务架构对DevOps的影响